

Scalable Data Loading and Preprocessing with `tf.data` and `TfRecord`

A reproducible end-to-end pipeline on the California housing data

Xaysomvang (Tom) Thammavong

github.com/ToMooGo/scalable-data-loading-tfdata

Technical report. All numbers in this report are generated by the run described in Section 11: configuration `configs/full_flow_config.yaml`, seed 42, data SHA-256 prefix `8a3727f4cf54`.

Summary of findings

This report builds Chapter 13 of Géron’s book [1], on loading and preprocessing data, as one runnable system on the California housing data, and measures what each tool does on a machine with two CPU cores. The six findings below are those of the run described in Section 11. The first three are negative.

1. On data this small, a computation in memory is faster than every streaming pipeline. Reading the CSV shards into a pandas data frame and slicing batches with NumPy delivers 354,718 examples per second, against at most 25,885 for the CSV pipelines and 62,195 for the TFRecord pipeline (Sections 4 and 5). What the streaming pipeline does show is how memory grows: with the training set repeated 100 times, the memory of the `tf.data` pipeline grows by 31 MB and that of pandas by 182 MB.
2. TFRecord does not save space for these data. An uncompressed TFRecord file is 4.17 times as large as the CSV, and the GZIP-compressed TFRecord is 1.68 times as large as the GZIP-compressed CSV (Section 5).
3. The speed-ups of the book’s steps appear only when the automatic optimisations of `tf.data` are switched off, and AUTOTUNE did not help on two cores. With only the listed steps, the book’s function reads 1.54 times as fast as the plainest pipeline; with the `tf.data` defaults and a training step of 4 ms the ratio is 0.98. Replacing the book’s fixed thread counts and `prefetch(1)` by AUTOTUNE gave 0.75 times the speed of the book’s function, and the AUTOTUNE variants of Part D were slower than their serial counterparts in all 8 cases (Sections 4 and 7).
4. Parsing a whole batch of records in one call is what makes the TFRecord pipeline fast: it reads 3.3 times as fast as parsing one record at a time and 3.6 times as fast as the CSV pipeline. A CSV reader that parses a batch per call was not tested, so this factor does not separate the file format from the batching (Section 5).
5. The 400-cell grid of latitude and longitude helps the linear model clearly and the small neural network little. For the linear model it lowers the validation RMSE by 6.7%. The network is already 23.6% better than the linear model without it, and of the 8 richer feature sets the paired intervals show 0 to be better than the eight numbers alone (Section 6).
6. The deployed model, the network with the hashed grid, has a test RMSE of \$50,751, with a 95% interval from \$48,690 to \$52,732 computed as in Chapter 2 from the 4,128 test rows; this is 24.7% below the linear model of Chapter 2. Its error is largest on the 179 test rows at the largest value, \$500,001, where the book notes that the values are capped (an RMSE of \$92,718, against \$47,987 on the other rows). The largest difference between the predictions of the request path and of the training path is 0 dollars (Section 8).

The formulas of Section 9 for the size of a TFRecord file and for the order left by a shuffle buffer agree with the measurements, and the formula for prefetching has the measured shape. Section 10 lists what is not claimed.

Contents

1	Introduction	3
2	System overview	3
2.1	Flows	4
2.2	The pipeline that the book describes	5
2.3	Tests and continuous integration	5
3	Data and preparation	5
4	Part A: the Data API on sharded CSV files	7
4.1	The pipeline	7
4.2	Speed	8
4.3	Prefetching and the formula	9
4.4	How well a shuffle buffer mixes sorted data	11
5	Part B: the TFRecord format	11
5.1	From CSV to TFRecord	11
5.2	The size of a file can be computed	12
5.3	Size and reading speed	13
5.4	Damaged files are detected	14
5.5	Scaling up	15
6	Part C: preprocessing inside the model	16
6.1	From the Features API to Keras layers	16
6.2	The features	16
6.3	Feature sets and the experiment	17
6.4	Results	18
7	Part D: TensorFlow Datasets and images in TFRecord files	21
7.1	<code>tfds.load</code>	21
7.2	MNIST stored as TFRecord files	21
7.3	The same classifier from both sources	23
8	Part E: the deployed model	23
8.1	The model	23
8.2	Quality on the test set	24
8.3	Quality gates	25
8.4	Serving and monitoring	26
9	Mathematical notes	28
9.1	Prefetching and the time per batch	28
9.2	How well a shuffle buffer mixes sorted data	28
9.3	Size of a TFRecord file	29
9.4	Hashing the location grid into buckets	30
9.5	An embedding is a one-hot vector times a matrix	31
9.6	Comparing feature sets with paired seeds	31
10	Limits and what is not claimed	32
10.1	What I learned and what I would do next	33
11	Reproducing the results	34

1 Introduction

Training data rarely fits in memory in one piece, and a training loop that waits for its input wastes the hardware that it runs on. Chapter 13 of Géron’s *Hands-On Machine Learning* [1] addresses this with four tools: the `tf.data` Data API, which streams, shuffles, transforms and batches data from many files at once; the TFRecord format, which stores examples as serialised Protocol Buffers; the Features API, which turns raw columns into numbers inside the model; and the TensorFlow Datasets (TFDS) project, which downloads and prepares standard benchmark data sets.

This project builds that chapter as one complete, runnable system and measures what each tool does. It has five parts, one for each question below.

Part A. How is a CSV data set that is split into many files read as a stream of shuffled, standardised batches, and how much does each step of that pipeline (interleaving several files, a shuffle buffer, parallel parsing, prefetching) contribute to the speed and to the quality of the shuffling?

Part B. How are the same rows stored as a GZIP-compressed TFRecord, how large is the file compared with the CSV, can its size be predicted exactly from the Protocol Buffers format, and does the format detect a damaged file?

Part C. How are a one-hot encoding of a five-category column, a 20 by 20 grid of latitude and longitude crossed into 400 cells, and learned embeddings built as part of the model, and does each of them make the predictions better?

Part D. How does `tfds.load` deliver a benchmark data set (MNIST) as ready batches, and how does that compare with images stored by hand in TFRecord files?

Part E. How is the resulting model packaged, registered, checked against quality gates and served, so that the preprocessing that was used in training is the preprocessing that is used for a request?

The data are the California housing data of Chapter 2 of the same book. A data set of this size (20,640 rows) fits in memory many times over, so the project does not claim that streaming is necessary for it. What it does is to build the pipeline that scales, to measure its behaviour on a machine with two CPU cores, and to repeat the training set up to 100 times to see how memory and speed grow when the data stop being small.

Everything is generated by one command from one configuration file. The numbers in the text, the tables and the figures of this report are written by that run: the tables are converted to $\text{\LaTeX}$ by a script and the figures are drawn from the same tables, so the numbers of the tables and of the text cannot drift from the results. The numbers typed by hand are facts of the data, settings of the configuration and the run time of about 45 minutes. Section 2 describes the system, Section 3 the data, Sections 4 to 8 the five parts, Section 9 the mathematics that the experiments test, and Section 10 what is not claimed.

2 System overview

Figure 1 shows the components. The code that Chapter 13 describes lives in a Python package, `tfdata_mlops`. Around it there are the services that make it a system rather than a script: Prefect runs the workflows, MLflow records every run and holds the model registry, a FastAPI service serves the registered model together with a small web page, and a PostgreSQL database keeps the predictions that the service has made. The structure of this wrapper follows the public

example repository [2]; the content that is being wrapped, the data loading and preprocessing, comes from the book.

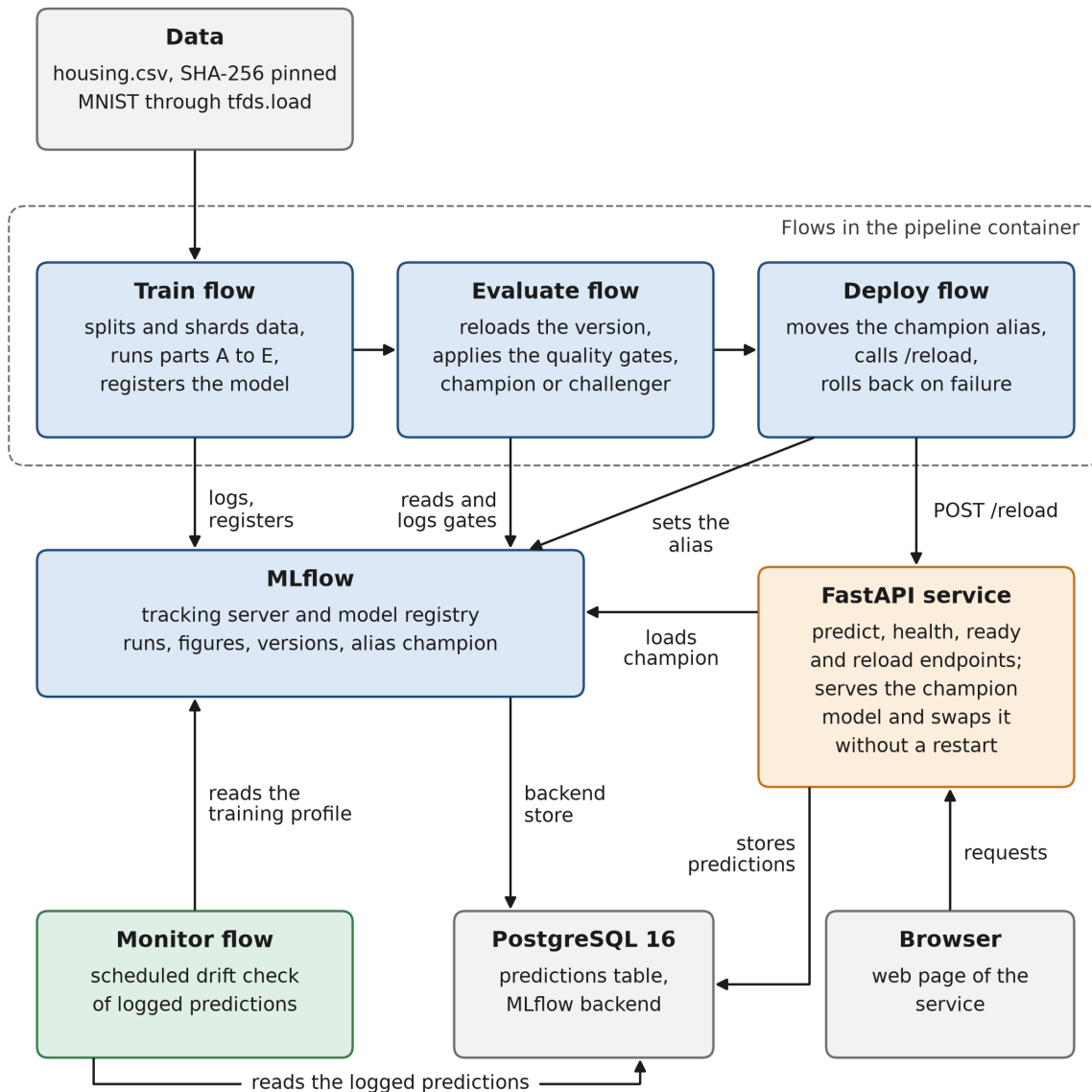


Figure 1. Components of the system. Each arrow points from the component that calls to the component that it uses; the arrows between the three flows show the order in which they run.

2.1 Flows

Four Prefect flows share one configuration file.

The *train flow* prepares the data once, then runs Parts A to E as separate tasks. Every part writes its parameters, metrics and tables to its own nested MLflow run under one parent run. The model of Part E is registered in the MLflow Model Registry together with its lineage: the SHA-256 of the data, the MD5 of the configuration and the git commit (recorded as unknown for the run of this report, which was made outside a git checkout). At the end the flow writes the result tables, the figures and a summary of the run to the **reports** folder.

The *evaluation flow* loads the registered candidate and re-evaluates it on the test rows that the training run had held out. A candidate passes only if it passes every one of the eight checks of Table 18: five numerical thresholds, two checks of behaviour, and the comparison with the current champion, which may be at most a stated tolerance worse. The *deploy flow* then gives the passing version the alias **champion** and asks the running service to reload it. The service must then report the new version and answer one canary request, a prediction for a held-out block. If it does not, the alias and the service return to the previous model, and a failed first deployment removes the alias again. When the service cannot be reached at all, the alias stays and the service loads it at its next start or registry check, unless the configuration requires the service, as it does in Docker; then the deployment is rolled back as well. Started alone, the deploy flow evaluates the candidate first, so a version that has not passed the checks is never promoted. The *monitor flow* reads the logged predictions and reports requests that are unlike the training data: unseen categories, values outside the training range, missing values and shifts of the mean measured in training standard deviations.

2.2 The pipeline that the book describes

Figure 2 shows the two data paths of Parts A and B; the models of Parts C and E are trained from the second one. A set of shard files is listed in random order, and five files are open at the same time, their lines interleaved one at a time (the book’s `n_readers=5`). Whether these files are read by several threads is a separate setting, `n_read_threads`: it is `None` in the book’s function, so the five files are read in turn, and `AUTOTUNE` in the pipelines that train the models of Parts C and E. A shuffle buffer mixes the lines. Each line is then parsed and standardised, the lines are collected into batches and the next batch is prepared while the current one is used. The right lane is the same idea on TFRecord files: each file is a GZIP-compressed sequence of serialised `tf.train.Example` messages, which can be parsed one batch at a time instead of one record at a time.

2.3 Tests and continuous integration

The package has a test suite that runs without network access or a GPU: it checks the pipeline against pandas, the TFRecord size formula, the damaged-file detection, the equality of a dense layer on one-hot input with an embedding, the quality gates, the drift checks, the deploy flow (canary request and rollback) and the API (validation, reload and rollback). A GitHub Actions workflow runs the style checks and the tests on two Python versions, and then builds the Docker images, starts the stack, runs the whole flow with a reduced configuration and sends a request to the service. Section 11 gives the commands.

3 Data and preparation

The data are the California housing census data that Chapter 2 of the book downloads. The file has 20,640 rows, one per census block group. There are eight numeric columns (longitude, latitude, median age of the houses, total rooms, total bedrooms, population, households and median income), one categorical column, `ocean_proximity`, with the five values `<1H OCEAN`, `INLAND`, `ISLAND`, `NEAR BAY` and `NEAR OCEAN`, and the target, `median_house_value` in dollars. The column `total_bedrooms` has 207 missing values. The file is taken from the book’s own repository and is checked against two pinned SHA-256 digests (the archive and the CSV inside it), so a changed download is refused.

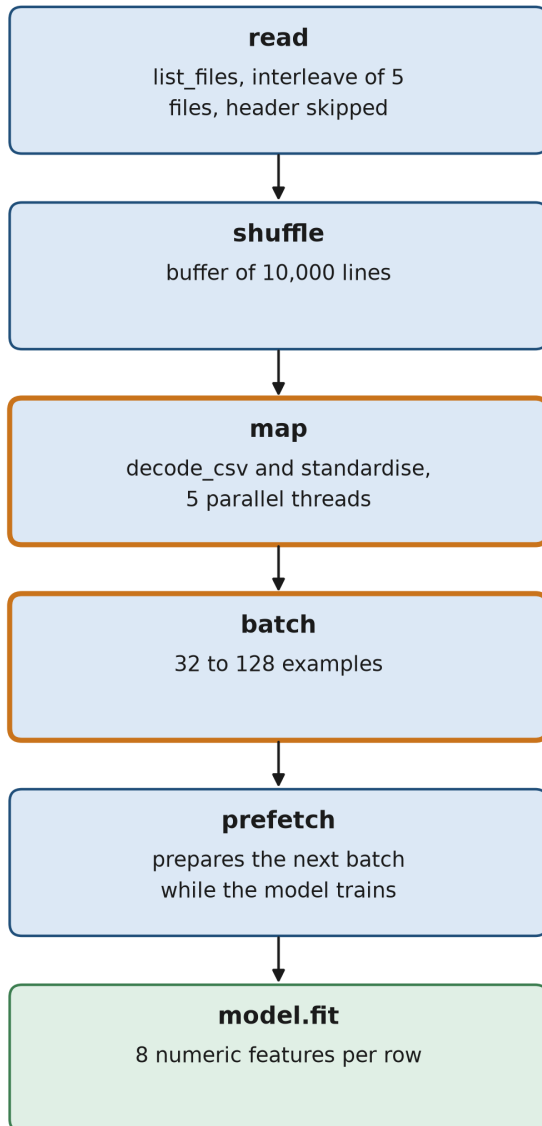
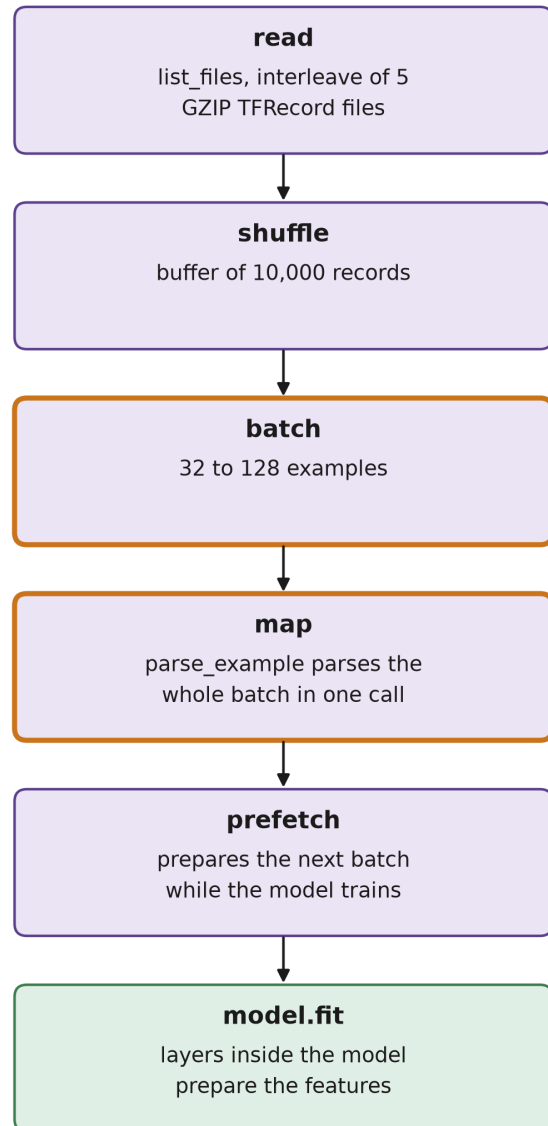
Part A: sharded CSV files**Part B: GZIP-compressed TFRecord files**

Figure 2. The two input pipelines. The left lane is the CSV pipeline of Part A and the right lane the TFRecord pipeline of Part B. Both end in `model.fit`. Orange edges mark the two steps that swap places between the lanes. Prefetch prepares the next batch in the background while the training step works on the current one.

Chapter 2 of the book notes that the median house value, which is the target here, was capped, and its scatter plot of income against value shows the cap as a horizontal line at \$500,000. Of the 4,128 test rows of this project, 179 (4.3%) have the largest value that occurs, \$500,001. These rows are kept, and Section 8 reports the error on them separately.

Split. The rows are split as in Chapter 2: stratified on five income categories, so that all three parts have the same income distribution as the whole. First, 20% of the rows, drawn at random within each income category, are held out as the test set. Of the remaining rows, 25% are drawn in the same way as the validation set; this second split is a choice of the project and not of the book. The result is a 60/20/20 split with 12,384 training, 4,128 validation and 4,128 test rows. The validation set is used to compare feature sets in Part C; the test set is used only for the final model and for the quality gates.

Statistics from the training rows only. A missing number is replaced by the training median of its column, as in Chapter 2, and numeric columns are standardised with the training mean and standard deviation. These statistics are computed once from the training rows and are never recomputed from the validation or the test rows. The target is divided by 100,000, so the model predicts hundreds of thousands of dollars and all errors are reported in dollars after multiplying back.

Shards. The book recommends splitting a large file into several files of equal length, because files of equal length interleave best. The training rows are written to 20 CSV files, the validation rows to 10 and the test rows to 10, and the same rows are written in the same order to uncompressed and to GZIP-compressed TFRecord files. A shard never mixes rows of different splits. Floating-point numbers are written with their full precision and a missing value is written as an empty field, so that the CSV loses no information and the pipeline’s own handling of missing values (Section 4) is exercised on real gaps.

4 Part A: the Data API on sharded CSV files

4.1 The pipeline

The function `csv_reader_dataset` follows the one in the book. It lists the files in random order and repeats them, keeps five files open at the same time and interleaves their lines, one line from each file in turn (the book’s `n_readers=5`; each file’s header is skipped), shuffles the lines in a buffer, parses each line with `tf.io.decode_csv`, collects batches and prefetches. In the parsing step an empty field is replaced by the training median of its column, through the `record_defaults` of `decode_csv` (the book’s defaults are 0; the median is a choice of this project that follows the imputer of Chapter 2), and the eight numbers are standardised with the training mean and standard deviation. The function has an argument for every step, so that each step can be switched off and measured. Reading threads are a separate argument, `n_read_threads`; the book notes that by default `interleave()` does not use parallelism, and its function leaves this argument at `None`, as do pipelines 1 to 5 below.

The first test is that the pipeline returns the same numbers as a plain pandas and NumPy computation of the same quantities. On the 12,384 training rows, of which 124 have a missing number, the largest absolute difference is 2.9×10^{-6} in the inputs and 2.4×10^{-7} in the targets, which is consistent with the rounding of 32-bit floating-point numbers.

4.2 Speed

Each timing is the median of 5 passes over the training rows (387 batches of 32) after one warm-up pass. To see what a consumer that is not instantaneous does to the pipeline, each batch is followed by a simulated training step of 0 ms (reading only) or 4 ms. Two settings of `tf.data` are compared. In the first, *only the listed steps*, the automatic graph optimisations of `tf.data` are switched off, so that a step is present only if the pipeline lists it. In the second, *tf.data defaults*, they are left on. The first answers the question “what does this step do”, the second “what does a user get”.

Table 1 and Figure 3 give the speed of six pipelines, each adding one step to the one above, and of the same computation done with pandas and NumPy. Each pass of this baseline reads the CSV shards into one data frame, standardises it and slices shuffled batches, so it holds all of the data in memory but also pays for reading and parsing the files.

Pipeline	No training step		4 ms training step	
	Only listed steps	tf.data defaults	Only listed steps	tf.data defaults
1. one file, no shuffle, serial parse	13,412	21,484	4,826	6,822
2. + interleave 5 files	15,632	19,019	4,792	6,851
3. + shuffle buffer	16,039	19,189	4,969	6,711
4. + 5 parse threads	18,935	25,885	5,275	6,686
5. + prefetch(1), the book’s function	20,606	19,168	6,794	6,692
6. + AUTOTUNE threads and prefetch	15,525	15,104	6,646	6,612
pandas and NumPy: read the shards, standardise, slice batches	354,718	354,718	7,530	7,530

Table 1. Examples read per second, without a training step and with a training step of 4 ms per batch. Row 5 is the function of the book.

With only the listed steps and no training step, the pipeline that reads one file at a time and parses serially reads 13,412 examples per second. Interleaving five files raises this to 15,632, the shuffle buffer to 16,039, five parsing threads to 18,935, and `prefetch(1)`, which completes the function of the book, to 20,606. The function of the book is therefore 1.54 times as fast as the plain pipeline. The steps in between should not all be read as separate effects: each added step changes the speed by a factor of between 1.03 and 1.18, and the noise of the measurement is of that order: within this run the same configuration was timed twice (row 6 with the *tf.data defaults* in Table 1, and the CSV row of Table 5) and gave 15,104 and 16,602 examples per second, 10% apart. Differences of that size or less between rows are therefore not treated as effects.

A training step makes prefetching matter. With a step of 4 ms the pipelines without prefetching read between 4,792 and 5,275 examples per second, and `prefetch(1)` raises the speed by a factor of 1.29 to 6,794. The ceiling for a step of 4 ms and batches of 32 is 8,000 examples per second, because the step alone takes that long; the function of the book reaches 90% of the speed of the pandas and NumPy computation, 7,530 examples per second, in the same situation.

Two results are not what the book’s description alone would lead one to expect. First, replacing the fixed values of the book (no reading threads, five parsing threads, `prefetch(1)`) by `tf.data.AUTOTUNE` for the reading threads, the parsing threads and the prefetching, with the same five interleaved files, did not help on this machine: without a training step it reads 15,525 examples per second, a ratio of 0.75 to the fixed values, and with a step of 4 ms the ratio is 0.98. The machine has two cores, and I did not investigate the cause further. Second, in the *tf.data defaults* setting the six pipelines are within 3.6% of each other when there is a training

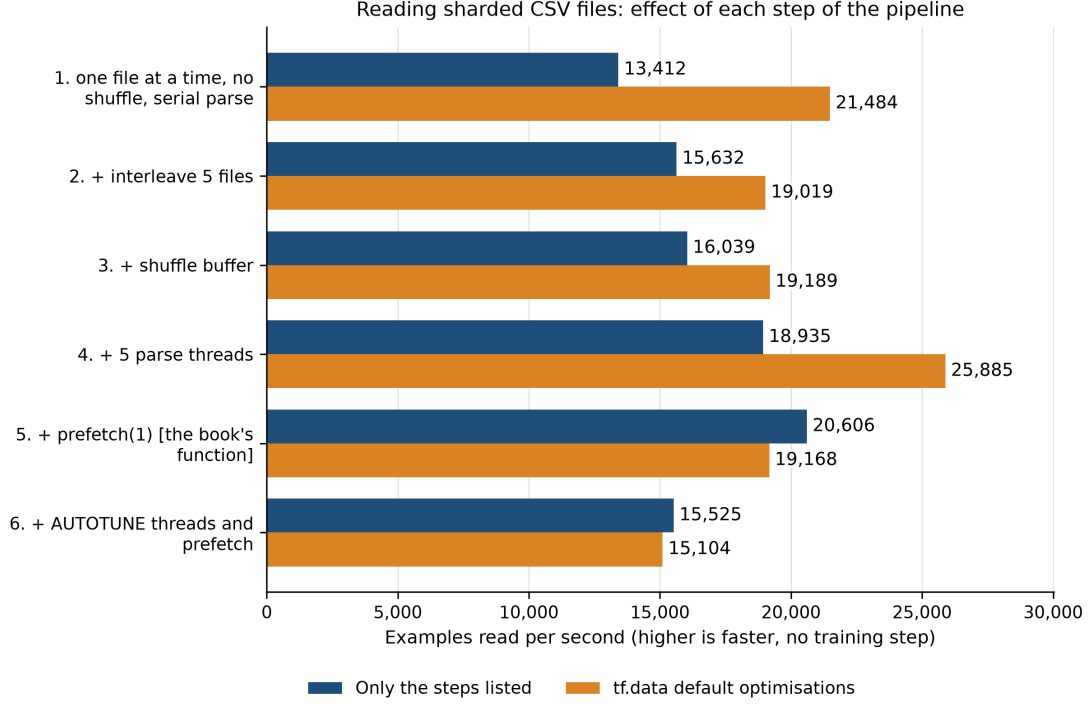


Figure 3. Reading speed of the six pipelines of Table 1, without a training step.

step (6,822 examples per second for the plainest and 6,692 for the function of the book), and without a step the values, from 15,104 to 25,885, do not form a monotone sequence. When the defaults are on, the plain pipeline is already as fast as the full one when there is a training step of 4 ms (a ratio of 0.98 for the function of the book to the plain pipeline), so the steps of the book show their effect when they are the only optimisation applied. This is why Table 1 reports both settings.

The pandas and NumPy computation is faster than any of the streaming pipelines when there is no training step: it delivers 354,718 examples per second, 17.2 times the function of the book. This is a property of the small data set, whose training rows take only about 1 MB. The point of the pipeline is not to beat a computation that loads everything into memory but to keep working when the data no longer fit (Section 5, Table 7).

4.3 Prefetching and the formula

Section 9 shows that with `prefetch(1)` the time per batch is $\max(t_{\text{in}}, t_s)$, where t_{in} is the time the input pipeline needs for a batch and t_s the time of the training step, and that without prefetching it is $t_{\text{in}} + t_s$. All of these runs read five interleaved files without reading threads and use a shuffle buffer of 10,000 lines, five parsing threads and no automatic optimisation (the *only listed steps* setting), with and without `prefetch(1)`. The time t_{in} is measured with no training step and the formulas are then compared with the measurements for steps from 0 to 8 ms (Table 2, Figure 4).

The shape of the formula is confirmed: with prefetching the time per batch stays at t_{in} as long as the step is shorter than t_{in} and then grows with a slope of one, and without prefetching it grows with a slope of one from the start. Without prefetching, the measured times exceed the formula by 0.38 to 0.59 ms per batch for steps of 1 ms and more. With prefetching they agree with t_{in} for steps of 0 and 1 ms (1.64 and 1.61 ms against 1.66 ms) and exceed the formula by

Step (ms)	No prefetch, measured	No prefetch, formula	prefetch(1), measured	prefetch(1), formula
0.0	1.66	1.66	1.64	1.66
1.0	3.05	2.66	1.61	1.66
2.0	4.04	3.66	2.66	2.00
4.0	6.11	5.66	4.74	4.00
8.0	10.25	9.66	8.76	8.00

Table 2. Time per batch in milliseconds, measured and from the formulas of Section 9, with t_{in} taken from the measurement with a step of 0 ms.



Figure 4. Time per batch against the length of the training step, with and without prefetching. The dashed lines are the formulas. For a step of 0 ms the two measured series are nearly equal, so their markers overlap.

0.66 to 0.76 ms for steps of 2 ms and more. Possible reasons, which I did not test, are the cost of handing a batch from one thread to the other and the fact that a simulated step that sleeps for a given time takes a little longer than that time. Because the excess is almost constant, the conclusion of the formula, that prefetching saves the smaller of the two times per batch, holds.

4.4 How well a shuffle buffer mixes sorted data

Shuffling is tested on the worst case, a training set stored in sorted order. The training rows are sorted by the target, written to 20 files, and read with different pipelines, in batches of 64. The order correlation is the Spearman correlation between the position of an example in the stream and its target: 1 means the stream is still sorted, 0 means that the order is unrelated to the target. The batch diversity is defined in Section 9: 0 means that every batch contains almost the same targets, 1 that every batch looks like the whole set.

Shuffling	Readers	Buffer	Spearman	Formula	Batch diversity
no shuffling	1	0	1.000		0.006
random file order only	1	0	−0.011		0.058
interleave 5 files	5	0	0.005		0.920
buffer 100	1	100	1.000	1.000	0.033
buffer 1000	1	1,000	0.965	0.963	0.267
buffer 10000	1	10,000	0.061		0.974
interleave 5 + buffer 100	5	100	0.005		0.916
interleave 5 + buffer 1000	5	1,000	0.006		0.943
interleave 5 + buffer 10000	5	10,000	−0.010		0.987

Table 3. Reading sorted data. “Formula” is the prediction of Section 9, which is compared with the measurement only for buffers of at most a quarter of the 12,384 rows (here 100 and 1,000). The rows with “random file order” or “interleave” also shuffle the order of the files; the rows “buffer” alone do not. “Readers” is the number of files interleaved (`n_readers`); none of these pipelines uses reading or parsing threads.

A buffer of 100 rows leaves sorted data almost sorted (correlation 1.000, formula 1.000), and a buffer of 1,000 rows leaves it mostly sorted (0.965 against the predicted 0.963). In the 2 cases in which the formula was compared with the measurement the largest difference is 0.002. A buffer of 10,000 rows is 81% of the data, beyond the quarter up to which the formula is compared, and reduces the correlation to 0.061.

The correlation alone can mislead. Reading the files in random order, with no buffer, gives a correlation of −0.011, which looks like perfect shuffling, but the batch diversity is only 0.058: each file is still sorted, so every batch holds almost the same targets. Interleaving five files at once, as the book does, gives a diversity of 0.920 already and, together with a buffer of 10,000 rows, 0.987. This is the reason for the two steps of the book: the shuffling of the file order and the interleaving mix the data coarsely, the buffer mixes it finely.

5 Part B: the TFRecord format

5.1 From CSV to TFRecord

A TFRecord file is a sequence of binary records, and each record here is one serialised `tf.train.Example`: a Protocol Buffers message that maps feature names to a list of floats, of integers or of byte strings. A housing row becomes an `Example` with ten features: the eight numbers and the target as float lists of one element, and `ocean_proximity` as a byte string. A missing number is not written. The rows are written in the same order and into the same number of files as the CSV shards, once uncompressed and once with GZIP compression (`tf.io.TFRecordOptions`).

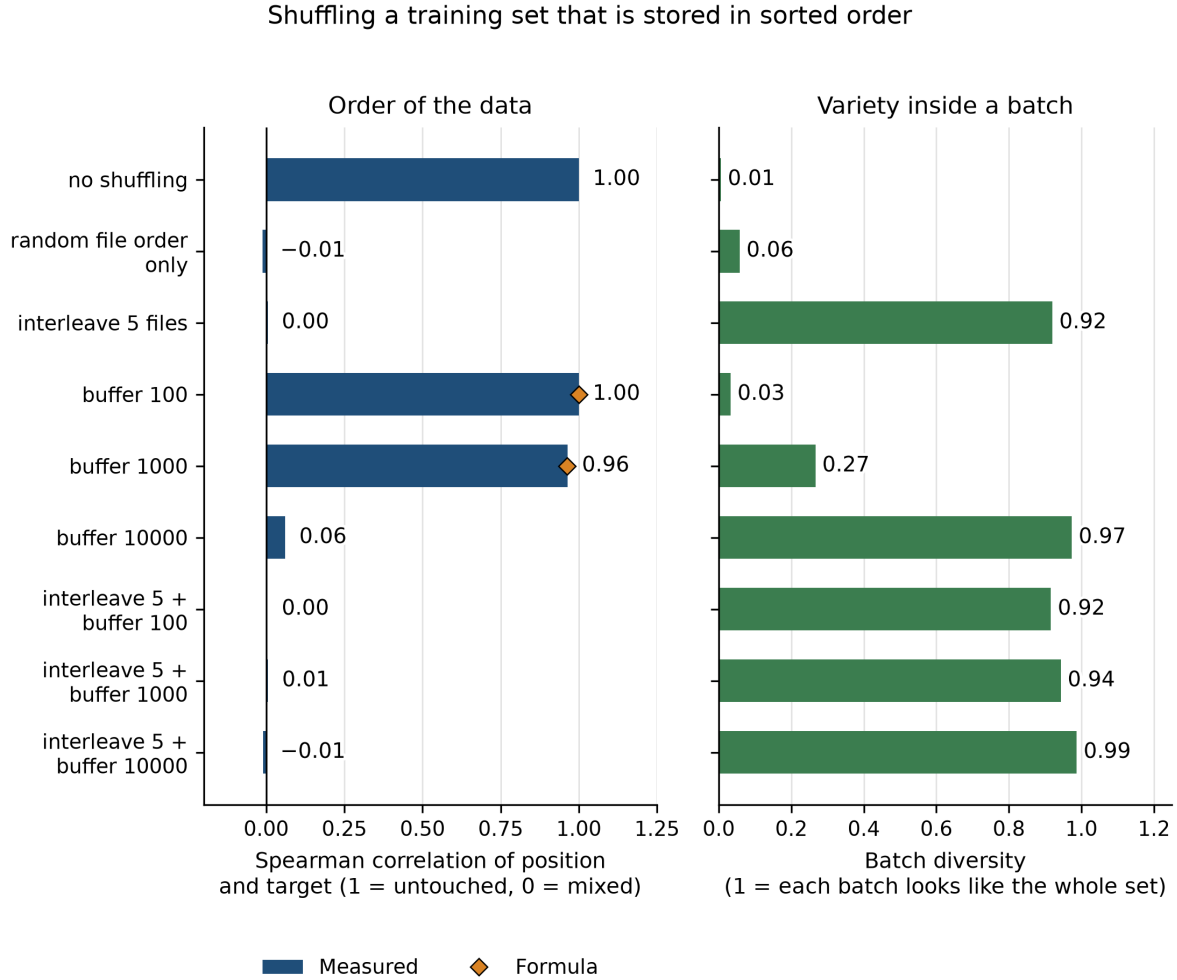


Figure 5. Order correlation (left) and batch diversity (right) for the pipelines of Table 3. The diamonds on the left are the predictions of the formula.

Reading uses `tf.io.FixedLenFeature` for every feature. A missing number then gets its *default value*, which is set to the training median, exactly as for the CSV pipeline; a test checks that the value that comes out of a record without `total_bedrooms` is the training median. The target has no default, so a record without a target raises an error.

The CSV and the TFRecord pipelines return the same data: the largest absolute difference between the numbers they produce is 0, the categories are equal, and the compressed files hold the same records as the uncompressed ones.

5.2 The size of a file can be computed

Section 9 derives the number of bytes of a serialised `Example` and of a file from the Protocol Buffers encoding. The experiment computes this number for each of the 12,384 training rows from its values alone, without writing anything, and compares it with the real length. Table 4 shows that every record, and therefore the whole file, has exactly the predicted size. The records have between 243 and 275 bytes; the differences come from the missing numbers, which are left out, and from the length of the category name.

Quantity	Value
Records	12,384
Serialised Example, smallest and largest (bytes)	243 and 275
Serialised Example, mean (bytes)	272.79
File size from the formula (bytes)	3,576,363
File size on disk (bytes)	3,576,363
Formula equals the length of every record	yes

Table 4. The size of the uncompressed TFRecord file of the training rows, predicted and measured.

5.3 Size and reading speed

Table 5 and Figure 6 compare five ways of storing and reading the same 12,384 training rows: the CSV shards, the same shards compressed with GZIP, TFRecord shards parsed one record at a time (`tf.io.parse_single_example`), TFRecord shards parsed a batch at a time (`tf.io.parse_example` after `batch`), and the compressed TFRecord shards parsed a batch at a time. All five use the same settings: five interleaved files, a shuffle buffer of 10,000 examples, `tf.data.AUTOTUNE` for the reading and parsing threads and for prefetching, one pass and no training step.

Format and reading	Bytes	Size vs CSV	Examples per second
csv	856,778	1.00	16,602
csv.gz	287,248	0.34	16,405
tfrecord, parse one record at a time	3,576,363	4.17	17,964
tfrecord, parse a batch at a time	3,576,363	4.17	60,155
tfrecord.gz, parse a batch at a time	483,662	0.56	62,195

Table 5. Size and reading speed of the same training rows in five formats.

Size. An uncompressed TFRecord is 4.17 times as large as the CSV, because every record repeats the names of its ten features and adds the framing and the type information of the message, whereas a CSV states the names once in the header. Compression removes this repetition: the GZIP TFRecord is 0.56 times the size of the CSV. The comparison with the CSV is not the fair one, though. A GZIP-compressed CSV is only 0.34 times the size of the CSV, so the compressed TFRecord is 1.68 times as large as the compressed CSV. TFRecord does not win on size for these data, and the report does not claim that it does.

Speed. Parsing one record at a time is as fast as reading the CSV (1.08 times the speed of the CSV, which is within the noise of Section 4). The gain comes from parsing a whole batch of serialised records in one call: it is 3.6 times as fast as the CSV, and 3.3 times as fast as parsing record by record. The CSV pipeline parses one line per call; a CSV reader that parses a whole batch of lines with one call was not tested, so the factor of 3.6 does not separate the effect of the format from the effect of batching. All five rows use `tf.data.AUTOTUNE`, which Section 4 found slower than fixed thread counts on this machine, and the ratios were not measured again with fixed counts. The compressed files are read as fast as the uncompressed ones (3.7 times the speed of the CSV), so on this machine the decompression is not a cost that shows in the measurement.

CSV and TFRecord shards holding the same training rows

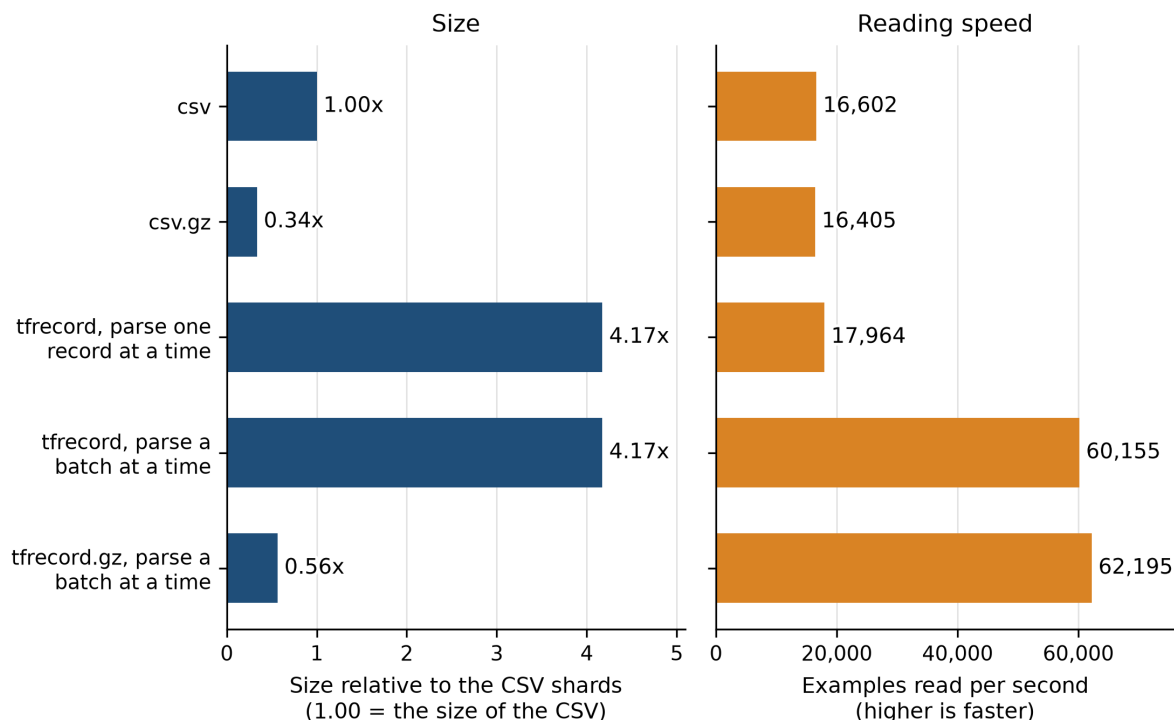


Figure 6. Size (left) and reading speed (right) of the five formats of Table 5.

5.4 Damaged files are detected

Every record carries two CRC checksums, one for its length and one for its data (Section 9). Table 6 shows what happens when a file that reads correctly is damaged in the middle (one byte flipped) or cut off at 60% of its length. In all 4 cases tested, for both the uncompressed and the compressed file, reading stops with a `DataLossError` instead of delivering wrong numbers. In the uncompressed file the flipped byte is caught by the checksums of the record; for the compressed file the test does not show whether the error came from the checksums of the records or from the decompression. This is the result of the run reported here and not a guarantee: the byte at which a file is cut depends on the order in which Protocol Buffers writes the features, and in about one test process in forty a compressed file that was cut at 60% was read to its end as a shorter file without an error. A pipeline that must not lose records therefore compares the number of records read with the number written, in addition to relying on the checksums.

File	Case	Result	Error raised
uncompressed	intact	read 620 records	no
uncompressed	one byte flipped in the middle	<code>DataLossError</code>	yes
uncompressed	truncated at 60%	<code>DataLossError</code>	yes
GZIP	intact	read 620 records	no
GZIP	one byte flipped in the middle	<code>DataLossError</code>	yes
GZIP	truncated at 60%	<code>DataLossError</code>	yes

Table 6. Reading an intact and a damaged shard of 620 records.

5.5 Scaling up

The training set is only about 1 MB, so the interesting question is what the two approaches cost when the data are larger. The training shards are repeated N times, for $N = 1, 10$ and 100 , giving up to 1,238,400 rows and 85.4 MB of CSV, and two ways of reading them are compared, each in a fresh process: one pass through the compressed TFRecord shards with the `tf.data` pipeline (shuffle buffer of 10,000 examples, batches of 32), and loading all CSV shards into a pandas data frame and standardising it. The memory growth is the increase of the resident memory of the process during the pass.

Copies	Rows	CSV (MB)	TFRecord.gz (MB)	tf.data growth (MB)	pandas growth (MB)	tf.data pass (s)	pandas load (s)
1	12,384	0.9	0.5	11	8	0.2	0.0
10	123,840	8.5	4.7	26	34	1.9	0.2
100	1,238,400	85.4	47.2	31	182	20.5	1.1

Table 7. Memory and time of one pass over the training set repeated N times.

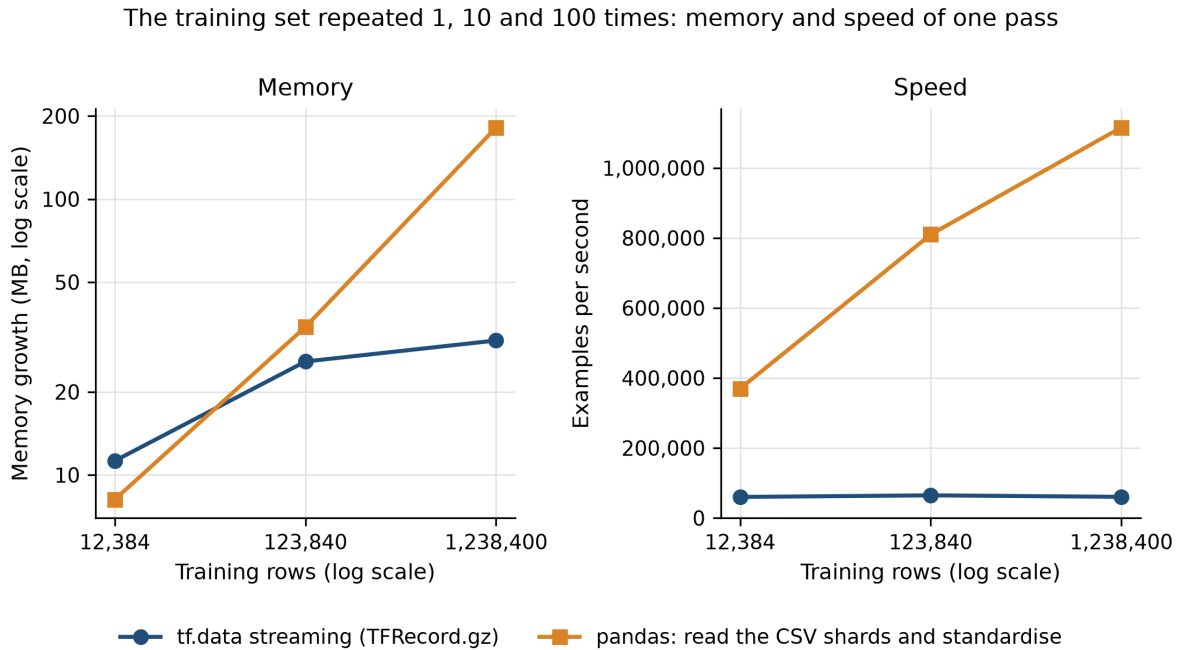


Figure 7. Memory growth (left) and reading speed (right) of the streaming pipeline and of pandas when the training set is repeated up to 100 times.

The memory of the streaming pipeline grows from 11 to 31 MB, while the 100-fold data set grows by a factor of 100; that of pandas grows from 8 to 182 MB, which is 5.9 times as much as the pipeline at the largest size, and it would keep growing with the data. The speed of the pipeline stays between 60,138 and 64,762 examples per second at every size. For the smallest data set the pipeline needs more memory than pandas; a plausible reason, which I did not test, is the fixed cost of the buffers and thread pools of `tf.data`. Two things must not be overlooked. Pandas is faster in time: loading the 1,238,400 rows takes 1.1 s, a pass of the streaming pipeline takes 20.5 s, and the two do not do the same work, since the pipeline also shuffles, parses and batches. And the pandas data frame at the largest size still fits in memory comfortably, so these data do not yet show the case in which the streaming approach is the only one that works. They

show how the two approaches scale: while the data grow by a factor of 100, the memory of the pipeline grows by a factor of 2.7 and that of pandas by a factor of 22 (factors computed from the unrounded measurements, so they differ slightly from the ratios of the rounded cells of Table 7).

6 Part C: preprocessing inside the model

6.1 From the Features API to Keras layers

Chapter 13 builds the features of a model with `tf.feature_column`: standardised numeric columns, a bucketised column, a categorical column with a vocabulary turned into an indicator (one-hot) column, a crossed column that is hashed, and an embedding column, all joined by a `DenseFeatures` layer. Keras 3 no longer contains `DenseFeatures`, so I built the same features with Keras preprocessing layers, which are part of the model (Table 8). The boundaries, the vocabulary, the 20 by 20 grid and the sizes of the hash tables (100 and 1,000 buckets) are the numbers of the book.

Book (<code>tf.feature_column</code>)	This project (Keras 3 layers)
<code>numeric_column</code> with <code>normalizer_fn</code>	Normalization with the training mean and variance
<code>bucketized_column</code>	Discretization
<code>categorical_column_with_vocabulary_list</code>	<code>StringLookup</code> ; index 0 is the bucket for unseen values
<code>indicator_column</code> (one-hot)	the one-hot output mode of the lookup or the discretisation
<code>crossed_column</code> (hashed)	<code>HashedCrossing</code>
<code>embedding_column</code>	<code>Embedding</code>
<code>DenseFeatures</code>	Concatenate of the blocks

Table 8. The Features API of the book and the layers that replace it.

Because the layers are part of the model, the model takes the raw columns, and the preprocessing that is used in training is the one that is used when the model is served. The book’s TF Transform section warns about the difference between the two (training/serving skew); here a quality gate checks that the request path and the `tf.data` path give the same predictions (Section 8). The layers that have no learned weights are also tested against a plain NumPy computation of the same features on the validation rows; the largest difference is 2.9×10^{-6} .

6.2 The features

Three ideas of the chapter are implemented.

One-hot encoding of a categorical column. `ocean_proximity` has five categories. A `StringLookup` layer with the five-word vocabulary turns it into a one-hot vector of six elements: the first element is the bucket for any category that the model has not seen, so that an unknown word at serving time gives an answer instead of an error.

A crossed grid of latitude and longitude. Latitude is cut at 19 equally spaced boundaries from 32 to 42 degrees, which gives 20 intervals of which the first and the last are open-ended, longitude in the same way into 20 intervals with boundaries from -125 to -114 degrees, and the two are crossed into a grid of $20 \times 20 = 400$ cells. Two forms are used. The exact grid is the outer product of the two one-hot vectors, a one-hot vector of 400 elements. The book’s form hashes the pair of indices into 1,000 buckets (with `HashedCrossing` here); the book says that the number of buckets must be high enough to avoid too many collisions and that a higher number needs more memory for the embedding table. With 400 cells in 1,000 buckets this form does not make the table smaller than the exact grid; I used it because it is the form of the book. Section 9 computes how many collisions to expect when 400 cells are hashed into 1,000 buckets.

Embeddings. An embedding replaces a one-hot vector by a short vector of learned numbers: of dimension 2 for the five categories, which is the value of the book, and of dimension 8 for the cells of the grid and 4 for the cross of age and category, which are choices of this project (age is cut into six intervals and crossed with `ocean_proximity`, hashed into 100 buckets). The embedding of the exact 400 cells is implemented as a `Dense` layer without a bias on the one-hot vector, which is the same thing (Section 9); the embedding of the hashed cells is an `Embedding` layer. Section 9 shows that an embedding is a one-hot vector multiplied by a matrix, and a test in the package checks that a dense layer without a bias on the one-hot input and an embedding give the same output.

6.3 Feature sets and the experiment

The nine feature sets of Table 9 are compared. Each set adds one idea to the baseline, the numbers plus the one-hot `ocean_proximity`, except the first, which has the numbers only, and the last, which combines the income buckets, the age cross and the hashed grid. Each set is used in two models: a linear model, which has no hidden layer, so that the quality of the features is visible without a network that could construct features itself, and a small neural network with two hidden layers of 64 units. Every combination is trained with 5 seeds, with early stopping on the validation error, reading the GZIP-compressed TFRecord shards with `tf.data`. This makes 9 feature sets $\times$ 2 models $\times$ 5 seeds trainings. The comparison uses the validation set; the test error is shown for information and is not used to choose.

	Feature set	Inputs of the model
1	numeric only	the 8 standardised numbers
2	+ ocean one-hot	1, and the one-hot vector of <code>ocean_proximity</code> (baseline)
3	+ ocean embedding	1, and a 2-dimensional embedding of <code>ocean_proximity</code>
4	+ income buckets	2, and the one-hot vector of the income bucket (boundaries 1.5, 3, 4.5 and 6)
5	+ age $\times$ ocean	2, and a 4-dimensional embedding of the hashed cross of age and ocean (100 buckets)
6	+ grid one-hot (400)	2, and the one-hot vector of the 400 grid cells
7	+ grid embedding (400)	2, and an 8-dimensional embedding of the 400 grid cells
8	+ hashed grid (1000)	2, and an 8-dimensional embedding of the 400 cells hashed into 1,000 buckets
9	all features	2, the income buckets, the age cross and the hashed grid

Table 9. The nine feature sets.

Differences between feature sets are tested with paired seeds (Section 9): for each seed the two sets use the same seed for the shuffling of the data and for the initialisation, so the difference

of their errors removes part of the noise of training. A difference is called clear when the 95% interval of the mean difference does not contain 0.

6.4 Results

Tables 10 and 11 give, for each feature set, the mean validation RMSE over the 5 seeds, the paired difference from the baseline with its 95% interval, and the test RMSE; Figure 8 shows the test RMSE with its interval across seeds.

Feature set	Inputs	Params	Valid. RMSE	Diff. from baseline [95% interval]	Test RMSE	Test R^2
numeric only	8	9	69,626	+770 [+746, +793]*	68,473	0.640
+ ocean one-hot	14	15	68,856	baseline	67,490	0.651
+ ocean embedding	10	23	68,902	+46 [+2, +89]*	67,462	0.651
+ income buckets	19	20	67,980	-876 [-901, -850]*	66,604	0.660
+ age $\times$ ocean	18	419	68,580	-276 [-309, -243]*	67,149	0.654
+ grid one-hot (400)	414	415	64,226	-4,630 [-4,657, -4,602]*	62,546	0.700
+ grid embedding (400)	22	3,223	64,583	-4,273 [-4,371, -4,174]*	62,787	0.698
+ hashed grid (1000)	22	8,023	64,675	-4,182 [-4,352, -4,011]*	62,846	0.697
all features	31	8,432	63,211	-5,645 [-5,737, -5,553]*	61,316	0.712

Table 10. Linear model on each feature set. Errors in dollars. The baseline is + ocean one-hot; a negative difference is an improvement, and * marks an interval that excludes 0.

Feature set	Inputs	Params	Valid. RMSE	Diff. from baseline [95% interval]	Test RMSE	Test R^2
numeric only	8	4,801	52,303	-312 [-1,088, +464]	51,852	0.794
+ ocean one-hot	14	5,185	52,616	baseline	51,530	0.796
+ ocean embedding	10	4,941	52,180	-436 [-1,159, +287]	51,387	0.797
+ income buckets	19	5,505	53,079	+463 [-133, +1,059]	51,730	0.795
+ age $\times$ ocean	18	5,841	52,706	+90 [-488, +669]	51,814	0.794
+ grid one-hot (400)	414	30,785	52,759	+143 [-356, +642]	51,590	0.796
+ grid embedding (400)	22	8,897	52,230	-385 [-964, +193]	51,065	0.800
+ hashed grid (1000)	22	13,697	52,002	-614 [-1,191, -36]*	50,668	0.803
all features	31	14,673	52,803	+188 [-586, +961]	51,606	0.796

Table 11. Neural network with two hidden layers of 64 units on each feature set, in the same form as Table 10.

Linear model. The features matter for the linear model. The baseline has a validation RMSE of \$68,856. Crossing latitude and longitude into the exact grid of 400 cells lowers it by \$4,630 (6.7%), and the two grid forms that use 22 inputs instead of 414, the learned embedding of the 400 cells and the hashed grid, lower it by \$4,273 and \$4,182. A linear model can make the price depend on the position only linearly in latitude and longitude; the grid gives every cell its own coefficient. All features together lower the error by \$5,645 (8.2%). The income buckets (\$876) and the cross of age and category (\$276) help little; the one-hot category itself is worth \$770 against the numbers alone, and a 2-dimensional embedding of the category is almost as good as the one-hot vector (\$46 worse). For all 8 comparisons the interval excludes 0, but the intervals are very narrow, because a linear model trained with different seeds ends almost in the same place. They describe the variation caused by the seeds and not what another split of the data would change.

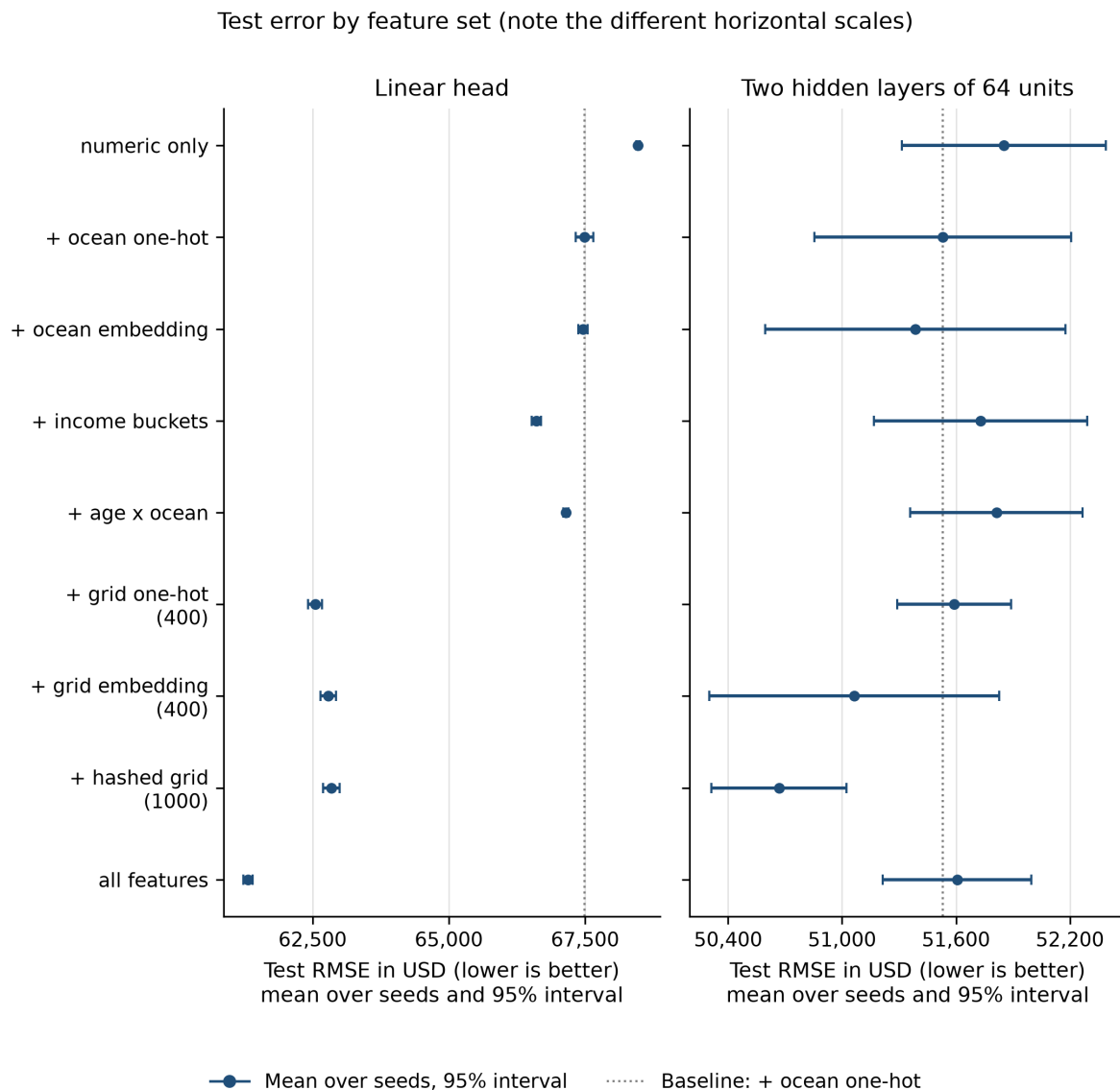


Figure 8. Test RMSE of the nine feature sets for the linear model (left) and the neural network (right): mean over the seeds with the 95% interval.

Neural network. With two hidden layers the baseline is already 23.6% better than the linear model (a validation RMSE of \$52,616), and the additional features change little. Only one of the 8 comparisons has an interval that excludes 0: the hashed grid improves the validation RMSE by \$614 (1.2%), with an interval from $-\$1,191$ to $-\$36$. All features together are not better than the baseline (a difference of $+\$188$, with an interval that contains 0). A plausible explanation, which I did not test separately, is that the hidden layers can build functions of position from raw latitude and longitude themselves, so that the grid adds less than it does for the linear model. The result of the single interval should be read with care (this is my own reading, not the book's): eight intervals of 95% were computed for this model, and one that only just excludes 0 is not strong evidence. The set with the lowest mean validation error of the neural network, **+ hashed grid (1000)**, is the one that is deployed (Section 8); the rule uses the validation set and not the test set. Compared in the same paired way with the network on the eight numbers alone, the number of the 8 richer feature sets whose interval shows them to be better is 0. Replacing the one-hot grid of 400 cells by an 8-dimensional embedding lowers the mean error from \$52,759 to \$52,230 and reduces the parameters from 30,785 to 8,897; the paired interval of this difference is

from $-\$955$ to $-\$102$, so on these data the smaller form was not worse. The paired comparisons of this paragraph and the next were chosen after looking at the tables, so the caveat about many intervals applies to them as well.

Hash collisions of the location cross. The 400 cells land in 335 distinct buckets out of 1,000, so 65 cells share a bucket with a cell that was there first. A uniformly random hash would be expected to use 329.8 buckets and to cause 70.2 such sharings (Table 12, Figure 9). Only 133 of the 400 cells contain training rows, since the data cover only a part of the rectangle that the grid spans, and of these just 6 share a bucket with another occupied cell; the training rows in these buckets are 0.76% of the data. In this project the same cells collided in repeated calls; I did not check other versions of Keras. The collisions were not isolated in a separate experiment, so I do not claim that they are harmless. For the neural network the hashed grid has the lower mean error ($\$52,002$ against $\$52,759$ for the exact grid, a paired interval of the difference from $-\$1,048$ to $-\$466$). For the linear model it is $\$449$ worse ($\$64,675$ against $\$64,226$, a paired interval from $\$271$ to $\$625$), and its mean is $\$92$ above that of the embedding of the exact cells, a difference whose paired interval, from $-\$10$ to $\$192$, contains 0.

Quantity	Value
Grid cells	400
Hash buckets	1,000
Distinct buckets used by all cells (measured)	335
Distinct buckets expected for a uniform random hash	329.8
Cells that contain training rows	133
Occupied cells that share a bucket with another occupied cell	6
Training rows in such shared buckets	94 (0.76%)

Table 12. Hashing the 400 grid cells into 1,000 buckets.

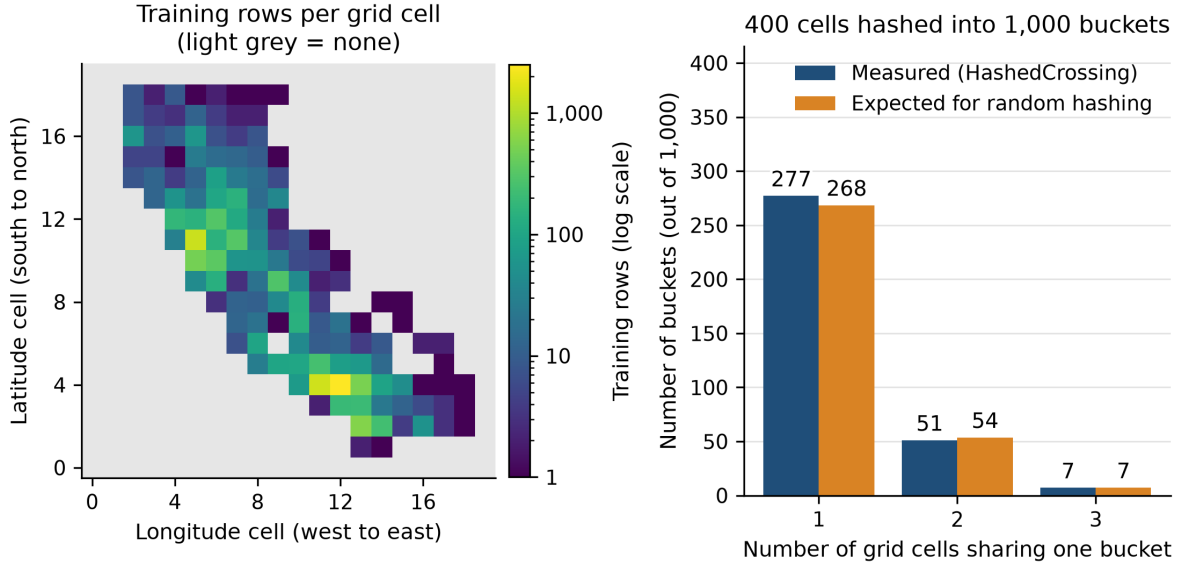


Figure 9. Left: the number of training rows in each cell of the 20 by 20 grid (light grey: none). Right: the number of buckets that hold k cells, measured and expected for a uniform random hash.

7 Part D: TensorFlow Datasets and images in TFRecord files

7.1 tfds.load

The last section of Chapter 13 loads MNIST and trains on it with a few lines (the model definition is elided in the book, as is its `compile` call):

```
dataset = tfds.load(name="mnist", batch_size=32, as_supervised=True)
mnist_train = dataset["train"].repeat().prefetch(1)
model.fit(mnist_train, steps_per_epoch=60000 // 32, epochs=5)
```

The project makes the first call with the same arguments. The result is a dictionary with a `train` and a `test` data set, each already batched, whose elements are a batch of images (28 by 28 pixels, one channel, unsigned bytes) and a batch of labels. The training set has 1,875 batches of 32 images, the number that the book’s `steps_per_epoch` gives for 60,000 images, and the test set has 313 batches, the last of which is not full because 10,000 is not a multiple of 32.

The version of TFDS used here downloads the four official MNIST files from a Google Cloud Storage address. Some firewalls and CI runners cannot reach that address, and this project was built in such an environment, so the project can fetch the same four files from a public mirror, compare their MD5 checksums with the official ones and hand them to the MNIST builder of TFDS; everything after the download is then the real `tfds.load` (Section 10). The route of this run was `mirror`.

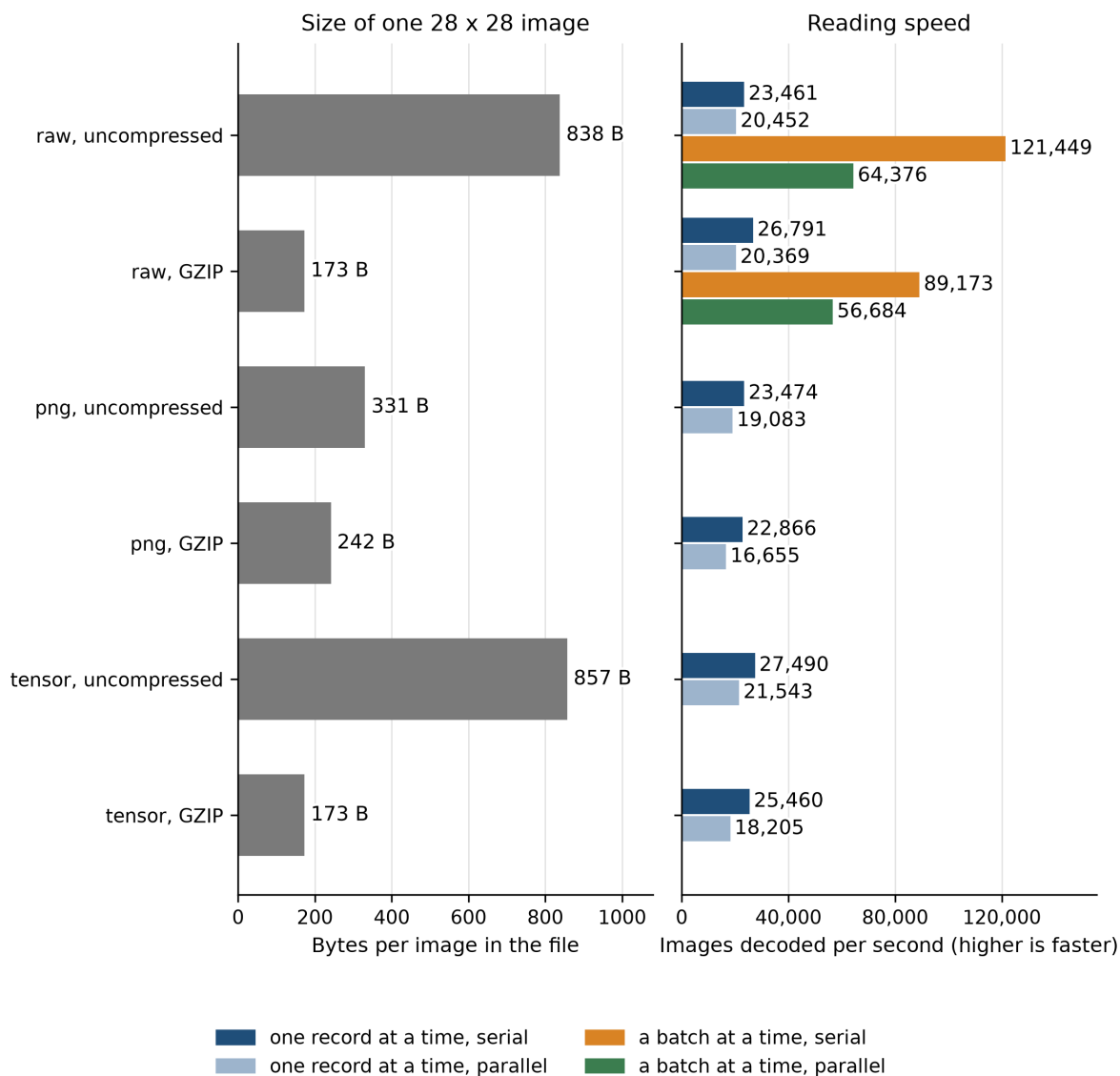
7.2 MNIST stored as TFRecord files

The book explains that a `BytesList` can hold any binary data: an image encoded with `tf.io.encode_jpeg`, or any tensor turned into a byte string with `tf.io.serialize_tensor`. Three encodings of the 60,000 training images are compared, each in 10 shards, once uncompressed and once with GZIP. The first is the serialised tensor of the book. The second is an encoded image; it is PNG and not the book’s JPEG, because PNG is lossless and the pixels that come back can be compared with the original ones. The third, the plain bytes of the pixel array, is an addition of this project (it is not in the book) and the only one that can be decoded for a whole batch at a time, because in the code of this project `tf.io.decode_raw` accepts a vector of strings, while `tf.io.decode_png` and `tf.io.parse_tensor` take one string. For all three encodings, the first 256 training images and their labels come back from the files exactly equal to the originals.

Encoding	Compression	Bytes/image	One record at a time		A batch at a time	
			Serial	Parallel	Serial	Parallel
raw	none	838	23,461	20,452	121,449	64,376
raw	GZIP	173	26,791	20,369	89,173	56,684
png	none	331	23,474	19,083	—	—
png	GZIP	242	22,866	16,655	—	—
tensor	none	857	27,490	21,543	—	—
tensor	GZIP	173	25,460	18,205	—	—

Table 13. Size and reading speed of the MNIST training images in three encodings. Speeds are in images per second, with no training step. “Parallel” uses `tf.data.AUTOTUNE` for reading, decoding and prefetching, “serial” uses none of these. A dash marks a combination that does not exist for that encoding.

The same MNIST training images stored in three encodings



Decoding a batch at a time is only possible for the raw encoding.

Figure 10. Size per image (left) and reading speed (right) of the six files of Table 13.

Size. The sizes are in Table 13 and Figure 10; the factors below are computed from the unrounded sizes. A raw record takes 838 bytes for 784 pixels. The 54 bytes in addition are the 16 bytes of framing of the record (Section 9) and the **Example** structure with the names **image** and **label**. The serialised tensor takes 19 bytes more than the raw bytes (857 bytes), which is the description of type and shape that it carries. A PNG image is 0.39 times as large as the raw bytes (331 bytes), because PNG compresses a picture that is mostly black. GZIP shrinks the raw and the tensor records to 173 and 173 bytes, factors of 4.9 and 5.0, and the PNG records to 242 bytes. The compressed raw record is therefore smaller than the PNG record, with or without GZIP on top of PNG. A plausible reason, which I did not test, is that GZIP is applied to the whole stream of records and can use what repeats from one record to the next, whereas PNG compresses one image at a time.

Speed. Decoding one record at a time in serial, the six files are read at between 22,866 and 27,490 images per second, which is a spread of 20%. Each configuration was timed in one experiment, and in Part A the same CSV pipeline timed in two experiments of one run differed by 10% (Section 4); I therefore do not draw a conclusion about which of the single-record encodings decodes fastest. The result that is large enough to be an effect is the one of Part B: decoding a whole batch of raw records in one call reads 121,449 images per second from the uncompressed files and 89,173 from the compressed ones, which is 5.2 and 3.3 times as fast as one record at a time (for the housing rows the factor was 3.3). As in Part B, the comparison mixes the effect of batching with that of the encoding, because the other two encodings could not be decoded for a batch at once. The AUTOTUNE variants were slower than their serial counterparts in all 8 cases, by factors between 0.53 and 0.87, in line with what Part A found for AUTOTUNE on this two-core machine; I did not investigate the cause.

7.3 The same classifier from both sources

A small classifier is trained from the two sources: the batches of `tfds.load`, used as in the book, and the compressed raw TFRecord files read with a shuffle buffer of 10,000 images and batch-wise decoding. The book elides the layers of its model and uses the optimiser `sgd`; the classifier here is a choice of the project (the pixels divided by 255, one hidden layer of 128 units, a softmax layer of 10 units, Adam), so its accuracy is not comparable with any number of the book. Both are trained for 5 epochs of 1,875 batches with the same 60,000 training images and with 3 seeds, the two sources sharing the random numbers of the initialisation for each seed. The accuracy is that on the test set after the last epoch; nothing was chosen with it.

Input	Seeds	Mean test accuracy	Standard deviation
<code>tfds.load</code>	3	0.9757	0.0010
TFRecord (raw, GZIP)	3	0.9764	0.0010

Table 14. Test accuracy of the same classifier trained from the two sources: mean and standard deviation over the seeds.

The mean accuracies (Table 14) are 0.9757 from `tfds.load` and 0.9764 from the TFRecord files, and all 3×2 values lie between 0.9748 and 0.9774. The paired difference (`tfds.load` minus TFRecord) has a mean of -0.0007 and a 95% interval from -0.0048 to 0.0035 , which contains 0. With three seeds the interval is wide, so this shows that no difference was detected and not that there is none. It is a check that the TFRecord files lead to the same classifier as the data of TFDS; the exact comparison of the first 256 images (0.4% of the 60,000) is direct evidence for those images only.

8 Part E: the deployed model

8.1 The model

The model that is deployed is the neural network with two hidden layers of 64 units on the feature set + `hashed grid` (1000). It takes the raw columns, so its preprocessing layers (the standardisation, the buckets, the hashed cross and the embeddings of Section 6) are stored in the same file as its weights. It is trained on the GZIP-compressed TFRecord shards with early stopping on the validation error: it ran for 55 epochs, the best one being epoch 45, and has 13,697 parameters. The validation RMSE is \$52,495; this is the run with seed 42 of Part C (the

same seed and the same data give the same result), whereas the \$52,002 of Table 11 is the mean of 5 runs, and the two differ by the variation between seeds.

The choice of the feature set is not itself a finding. With the caveat about many intervals of Section 6, the comparisons do not show any feature set to be better for the network than the eight numbers alone: the mean validation errors of the nine sets lie between \$52,002 and \$53,079, and the network on the numbers alone, with 4,801 parameters, is \$301 behind the deployed set, with a paired interval from $-\$294$ to $\$898$. The deployed set was chosen by the stated rule, the lowest mean validation RMSE, so its small advantage should not be read as an effect.

8.2 Quality on the test set

The test set was not used to choose the model, the feature set or any setting of the training; the thresholds of the quality gates were set after a first run (Section 10). Two yardsticks are used: always predicting the mean of the training targets, and the model of Chapter 2 of the book without its three combined attributes (median imputation, standardised numbers, one-hot category, linear regression) fitted on the same training rows.

	Test RMSE (USD)	95% interval (USD)
Always the training mean	114,164	
Linear model of Chapter 2	67,368	
Deployed model	50,751	48,690 to 52,732

Table 15. Error of the deployed model and of two yardsticks on the 4,128 test rows. The interval is computed for the deployed model, whose predictions are stored with the results, by the method of Chapter 2 (Section 9.6).

The deployed model has a test R^2 of 0.8024 and a mean absolute error of \$34,573. Its test RMSE is 24.7% lower than that of the linear model of Chapter 2 and 55.5% lower than that of the mean. As in Chapter 2 of the book, a 95% confidence interval for this error is computed from the squared errors of the test rows; it runs from \$48,690 to \$52,732. It describes only the sampling variation of the 4,128 test rows: the model, its seed and the split are held fixed.

Chapter 2 of the book notes that the median house values were capped (Section 3). On the 179 test rows at the largest value, \$500,001, the RMSE of the deployed model is \$92,718 and its mean error is $-\$56,040$; on the other 3,949 rows the RMSE is \$47,987.

Figure 11 shows the learning curve, the predictions against the actual values, and the error by area type and by income band; the two tables behind the last panels are Tables 16 and 17.

Group	Test rows	RMSE (USD)	Mean error (USD)
<1H OCEAN	1,859	52,387	+2,116
INLAND	1,289	34,881	+3,807
ISLAND	3	193,405	+2,647
NEAR BAY	443	57,556	-130
NEAR OCEAN	534	66,795	+4,043

Table 16. Test error by `ocean_proximity`. The mean error is the mean of prediction minus actual value.

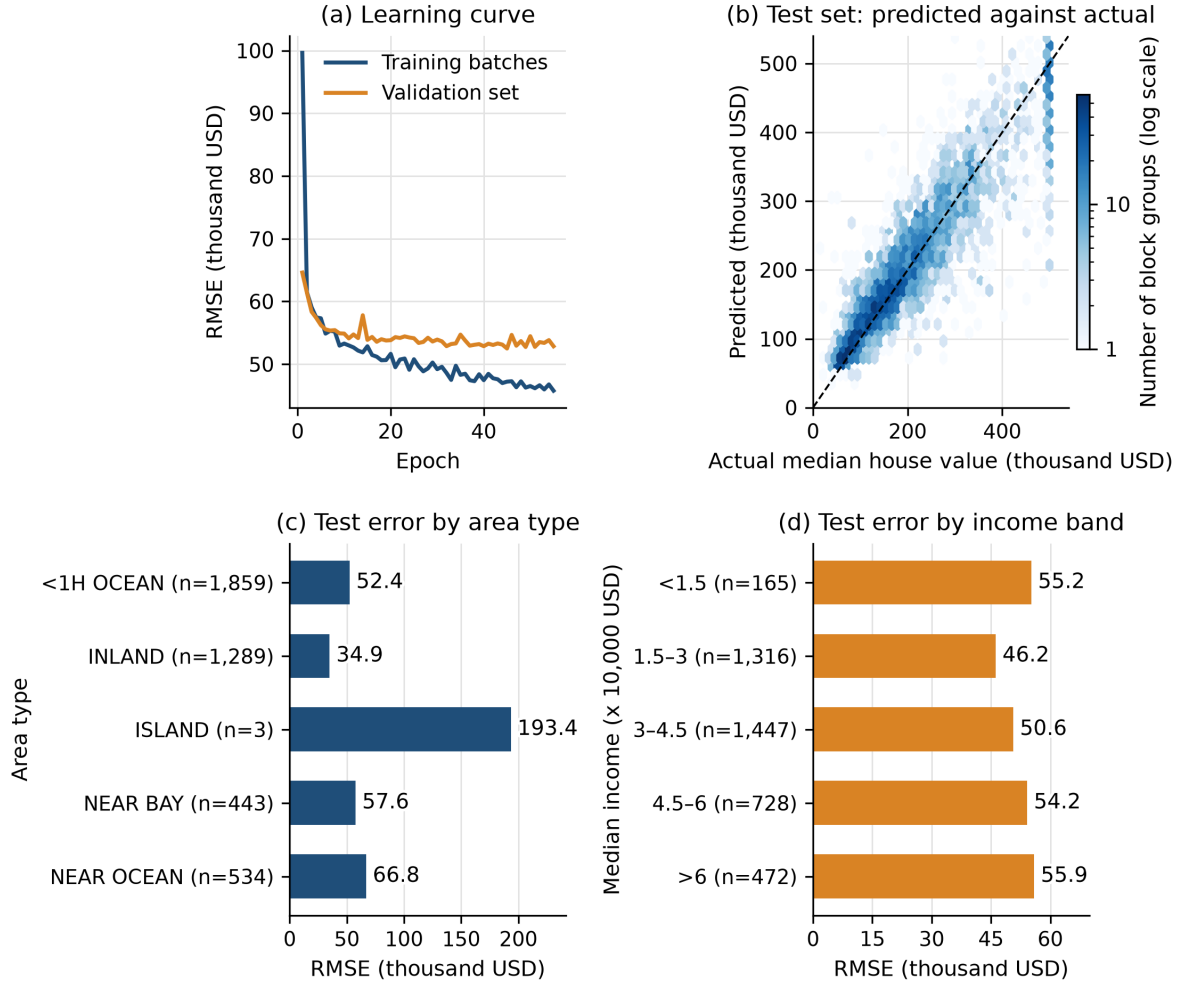


Figure 11. The deployed model. (a) RMSE on the training and validation data during training. (b) Predicted against actual value on the test set; the dashed line is a perfect prediction. (c), (d) RMSE on the test set by `ocean_proximity` and by income band.

Group	Test rows	RMSE (USD)	Mean error (USD)
<1.5	165	55,206	-2,673
1.5-3	1,316	46,202	+3,567
3-4.5	1,447	50,632	+1,667
4.5-6	728	54,156	+3,518
>6	472	55,914	+3,650

Table 17. Test error by income band (median income in tens of thousands of dollars).

8.3 Quality gates

A registered version is deployed only if it passes the checks of Table 18. The version is loaded from the registry, not taken from the training process, and evaluated on the test rows in two ways: through the *request path*, which sends raw rows to the same function that the API uses, and through the *pipeline path*, which reads the TFRecord shards with `tf.data`. Their largest difference is the training/serving skew; the gate requires that it is at most \$0.05. Further gates check that an unseen category gives a finite answer, that a missing number gives the same answer

as the training median in its place, that the registered file reproduces the error that the training run logged, and that the version is not worse than the current champion by more than 1%.

Check	Value	Required	Result
test RMSE (USD)	50,751	at most 60,000	pass
test R-squared	0.802	at least 0.750	pass
RMSE gain over the linear baseline (%)	25	at least 10	pass
training/serving skew, max difference (USD)	0.000	at most 0.050	pass
unseen category gives a finite answer	yes	yes	pass
missing number is replaced by the median	yes	yes	pass
reproduces the training RMSE, difference (USD)	0.000	at most 1.000	pass
test RMSE against the current champion (USD)	50,751	at most 51,758	pass

Table 18. Quality gates for the deployed version. The thresholds are choices of the project (Section 10), not content of the book.

The largest difference between the two paths on the test rows is 0 dollars, so there is no skew. A model that is saved, loaded again and used on all test rows gives predictions that are identical to the original (a largest difference of 0 dollars).

8.4 Serving and monitoring

The FastAPI service loads the model with the alias `champion` from the MLflow registry. Its endpoints are `/predict` for one row, `/predict/batch` for up to 1,000 rows, `/health` and `/ready` for the liveness and the readiness of the service, `/model-info` for the version and the profile of the training data (value ranges, categories, share of missing values, the location grid), and `/reload`, which the deploy flow calls to replace the model without restarting the service and which keeps the old model if loading the new one fails. Inputs are validated: impossible values are rejected, while values that are possible but outside the range of the training data are accepted and flagged in the answer, as are missing numbers and unseen categories. Every request is stored in PostgreSQL. The monitor flow reads these requests and reports the share of unseen categories, of values outside the training range and of missing values, and the shift of the mean of each input and of the predictions, measured in training standard deviations. The web page of the service (Figure 12) estimates a price, shows what the preprocessing layers made of the request (the standardised numbers and the cell of the location grid), displays the measurements of the training run, and shows the monitoring.

1 · Describe a census block

Raw columns only: the standardising, bucketing, crossing and embedding happen inside the deployed model.

longitude

-122.48

latitude

37.71

housing median age (years)

39

median income (× \$10,000)

5.1149

total rooms

3615

total bedrooms

632

population

1571

households

615

ocean proximity

NEAR OCEAN

Estimate price

OR LOAD A HELD-OUT BLOCK (▲ = DELIBERATELY DAMAGED COPY):

Held-out block

▲ Missing bedrooms

▲ Unseen area type

▲ Income out of range

2 · What the model sees

The 20 × 20 grid over California that the location cross is built on. Shaded cells held training blocks.

GRID CELL 225 (ROW 11, COLUMN 5) · INCOME BUCKET 4 OF 5

STANDARDISED NUMBERS (TRAINING STANDARD DEVIATIONS)

longitude		-1.45
latitude		+0.97
housing median age		+0.81
total rooms		+0.45
total bedrooms		+0.23
population		+0.13
households		+0.31
median income		+0.66

3 · Estimate and checks

\$382,005

actual \$314,200 · +\$67,805 (+22%)

✓ All values present, known area type, every number inside the training range.

model v2 · 14.69 ms · logged as request #81

Figure 12. The estimate page of the service. Top left: the form with the raw columns of a census block and the buttons that load a held-out block or a deliberately damaged copy of one. Top right: the cell of the location grid and the standardised numbers that the preprocessing layers made of the request. Bottom: the estimate, the actual value of the block, and the data-quality checks.

9 Mathematical notes

This section derives the formulas that the experiments use. Most of them are compared with a measurement in the sections above; Sections 9.5 and 9.6 are definitions and algebra, and the batch diversity is a definition.

9.1 Prefetching and the time per batch

Let t_{in} be the time the input pipeline needs to prepare one batch (read, shuffle, parse, batch) and t_s the time of one training step. Without prefetching the two alternate, so the time for N batches is

$$T_N^{\text{serial}} = N(t_{\text{in}} + t_s). \quad (1)$$

With `prefetch(1)` a background thread prepares batch k while the training step works on batch $k - 1$. Write p_k for the time at which batch k is ready and f_k for the time at which step k ends, with $f_0 = 0$. The model assumes that the thread starts preparing batch k as soon as the one-batch buffer is free, that is when step $k - 1$ starts (batch 1 starts at time 0, so $p_1 = t_{\text{in}}$), and that every batch takes exactly t_{in} . Step k starts when both the previous step has ended and batch k is ready, so

$$f_k = \max(f_{k-1}, p_k) + t_s. \quad (2)$$

Case $t_{\text{in}} \leq t_s$ (the step is the bottleneck). Batch k starts to be prepared when step $k - 1$ starts, which is at time $f_{k-1} - t_s$, and it is ready at $p_k = f_{k-1} - t_s + t_{\text{in}} \leq f_{k-1}$, that is before step $k - 1$ ends. Hence $p_k \leq f_{k-1}$ for $k \geq 2$, and the maximum is always f_{k-1} . Only the first batch has to be waited for: $f_1 = t_{\text{in}} + t_s$ and $f_k = f_{k-1} + t_s$, so

$$f_N = t_{\text{in}} + N t_s = t_{\text{in}} + (N - 1) t_s + t_s. \quad (3)$$

Case $t_{\text{in}} > t_s$ (the input is the bottleneck). The consumer is faster than the producer, so the buffer is never full and the producer never waits: $p_k = k t_{\text{in}}$. Step $k - 1$ ends at $f_{k-1} = (k - 1) t_{\text{in}} + t_s < k t_{\text{in}} = p_k$ (this is the induction hypothesis, and it holds for $k = 1$ with $f_0 = 0$), so each step starts the moment its batch is ready, $f_k = p_k + t_s = k t_{\text{in}} + t_s$, and

$$f_N = t_{\text{in}} + (N - 1) t_{\text{in}} + t_s. \quad (4)$$

Both cases have the form $f_N = t_{\text{in}} + (N - 1) \max(t_{\text{in}}, t_s) + t_s$. Dividing by N and letting N grow gives the time per batch

$$\lim_{N \rightarrow \infty} \frac{f_N}{N} = \max(t_{\text{in}}, t_s), \quad (5)$$

which is the overlap shown in the book's Figure 13-3. This is the dashed line in Figure 4, with t_{in} taken from the measurement with no training step.

9.2 How well a shuffle buffer mixes sorted data

A shuffle buffer of size B works as follows: it is first filled with the first B elements of the stream; then, for every output, one of the B elements in the buffer is chosen uniformly at random and sent out, and the next element of the stream takes its place. Suppose the stream is sorted, so the element with input index i has target rank i , for $i = 1, \dots, n$.

Consider an element that has just entered the buffer. At each later output it is the chosen one with probability $p = 1/B$, independently of the earlier outputs. The number G of outputs until it leaves therefore has a geometric distribution,

$$\Pr(G = g) = (1 - p)^{g-1}p, \quad \mathbb{E}[G] = \frac{1}{p} = B, \quad \text{Var}(G) = \frac{1-p}{p^2} = B(B-1). \quad (6)$$

The buffer holds the first B elements before the first output, and after m outputs it has taken in elements up to $B + m$. Element i can therefore first be chosen at output number $i - B + 1$ (for $i > B$), and it is output at position $(i - B + 1) + (G_i - 1)$, that is

$$O_i = i - B + G_i = i + \varepsilon_i, \quad \mathbb{E}[\varepsilon_i] = 0, \quad \text{Var}(\varepsilon_i) = B(B-1) \approx B^2, \quad (7)$$

where the start and the end of the stream, which behave slightly differently, are ignored.

The order correlation reported in the experiments is the Spearman correlation between output position and target. The positions O_i are not exactly ranks, so the Spearman correlation differs slightly from the Pearson correlation of i and O_i that is computed here (by about 0.003 for $B/n = 0.08$ in the experiment); I use the Pearson value as the approximation. Treat ε_i as independent of i . For i uniform on $1, \dots, n$,

$$\text{Var}(i) = \frac{n^2 - 1}{12} \approx \frac{n^2}{12}. \quad (8)$$

Then $\text{Cov}(i, O_i) = \text{Var}(i)$ and $\text{Var}(O_i) = \text{Var}(i) + B^2$, so

$$\begin{aligned} \rho &= \frac{\text{Cov}(i, O_i)}{\sqrt{\text{Var}(i) \text{Var}(O_i)}} = \frac{\text{Var}(i)}{\sqrt{\text{Var}(i) (\text{Var}(i) + B^2)}} \\ &= \frac{1}{\sqrt{1 + B^2/\text{Var}(i)}} \approx \frac{1}{\sqrt{1 + 12(B/n)^2}}. \end{aligned} \quad (9)$$

The approximation needs B much smaller than n : the noise ε_i is cut off by the ends of the stream when B is comparable to n , and a buffer as large as the data gives a uniformly random order, with $\rho = 0$. The report therefore compares the formula with the measurement only for $B \leq n/4$.

Batch diversity. The second quantity in Figure 5 is

$$D = \frac{\bar{s}}{s_{\text{all}}}, \quad \bar{s} = \frac{1}{K} \sum_{k=1}^K \text{sd}(y_{k,1}, \dots, y_{k,b}), \quad (10)$$

the mean standard deviation of the target inside a batch of b examples, divided by the standard deviation s_{all} of the whole training set. Batches that look like random draws have D close to 1; consecutive rows of sorted data have D close to 0. Gradient descent works best on the first kind of batch, which is why the book shuffles.

9.3 Size of a TFRecord file

The book gives the definition of the **Example** message and says that a record holds a length, a checksum of the length, the data and a checksum of the data. The byte counts below, the rules of the Protocol Buffers encoding, are not in the book; they are tested by Table 4, where the prediction equals the real size for every record.

A **tf.train.Example** is a Protocol Buffers message. A length-delimited field with a payload of L bytes occupies

$$F(L) = 1 + v(L) + L \quad \text{bytes}, \quad (11)$$

where the 1 is the tag (field numbers below 16 need one byte) and $v(L)$ is the number of bytes of L written as a varint: $v(L) = 1$ for $L < 128$, $v(L) = 2$ for $128 \leq L < 16384$.

A numeric feature holds a `float_list` with one 4-byte float, stored packed. The packed field is $F(4) = 6$ bytes, the `float_list` field around it is $F(6) = 8$ bytes, and this is the *Feature* message. A text feature of s bytes is a `bytes_list` holding one string, a Feature of $F(F(s))$ bytes. The features are entries of a map from a name to a Feature. An entry with a key of k bytes and a Feature of f bytes has the payload $F(k) + F(f)$, and itself occupies $F(F(k) + F(f))$ bytes in the map. With E the sum of the entry sizes over all features of one row, the serialised row is

$$S = F(E). \quad (12)$$

A missing number is left out of the message, which removes its entry from E .

A TFRecord file stores each serialised row as a header (8 bytes for the length and 4 bytes for the CRC of the length), the S bytes of data and a footer (4 bytes for the CRC of the data), so every record carries 16 bytes of framing besides S :

$$\text{file size} = \sum_{r=1}^R (S_r + 16). \quad (13)$$

The experiment computes this sum from the rows without writing a file and compares it with the real file size, byte for byte (Table 4).

A worked example. The nine numeric names have $9 + 8 + 18 + 11 + 14 + 10 + 10 + 13 + 18 = 111$ bytes in total, and the name `ocean_proximity` has 15. A numeric entry has a key of k bytes and a Feature of $f = F(F(4)) = 8$ bytes, so its payload is $F(k) + F(8) = (k + 2) + 10$ and the entry is $F(k + 12) = k + 14$ bytes. The nine numeric entries together have $111 + 9 \cdot 14 = 237$ bytes. The entry of the category, with a string of s bytes, has the payload $F(15) + F(F(F(s))) = 17 + (s + 6)$ and is $F(s + 23) = s + 25$ bytes. With all numbers present $E = 237 + s + 25 = 262 + s$, and because $E \geq 128$ the length of E takes two bytes, so $S = E + 3 = 265 + s$. For NEAR OCEAN ($s = 10$) this gives $S = 275$, the largest size in Table 4. For INLAND ($s = 6$) without `total_bedrooms`, whose entry has $14 + 14 = 28$ bytes, $E = 237 - 28 + 31 = 240$ and $S = 243$, the smallest size in the table.

The two CRCs are what detects a flipped byte in the uncompressed file; for the GZIP file the corruption test does not show whether the error came from these checksums or from the decompression.

9.4 Hashing the location grid into buckets

The 20 by 20 grid has $n = 400$ cells. The book hashes a crossed feature into $m = 1,000$ buckets; it notes that too few buckets cause collisions and that more buckets need a larger embedding table. If the hash behaves like a uniform random function, the number K of cells that land in one given bucket is binomial,

$$K \sim \text{Binomial}\left(n, \frac{1}{m}\right), \quad \Pr(K = k) = \binom{n}{k} \left(\frac{1}{m}\right)^k \left(1 - \frac{1}{m}\right)^{n-k}. \quad (14)$$

The expected number of buckets that hold exactly k cells is $m \Pr(K = k)$; this is the second set of bars in Figure 9. A bucket is empty with probability $(1 - 1/m)^n$, so the expected number of occupied buckets is

$$\mathbb{E}[\text{occupied}] = m \left[1 - \left(1 - \frac{1}{m}\right)^n\right] = 1000 [1 - 0.999^{400}] \approx 329.8, \quad (15)$$

(the number of occupied buckets is a sum of m indicator variables, each equal to 1 with probability $1 - (1 - 1/m)^n$, and the expectation of a sum is the sum of the expectations), so about $400 - 329.8 = 70.2$ cells are expected to share a bucket with an earlier cell. A given cell shares its bucket with at least one of the other 399 cells with probability $1 - 0.999^{399} \approx 0.33$. In this project the same cells collided in repeated calls of the `HashedCrossing` layer; I did not check other versions of Keras. What matters for the model is how many *training rows* fall in colliding cells, which the experiment reports because most of the 400 cells hold few rows or none.

9.5 An embedding is a one-hot vector times a matrix

Let a categorical feature take values $1, \dots, V$ and let $\mathbf{e}_j \in \{0, 1\}^V$ be the one-hot vector of value j (a 1 in position j , zeros elsewhere). An embedding of dimension d is a matrix $W \in \mathbb{R}^{V \times d}$ whose row j is the learned vector for value j . Multiplying the one-hot vector by the matrix selects that row:

$$\mathbf{e}_j^\top W = \sum_{i=1}^V (\mathbf{e}_j)_i W_{i,:} = W_{j,:}. \quad (16)$$

So a dense layer without a bias applied to a one-hot input is the same as an embedding lookup, and the lookup is cheaper because it reads one row instead of multiplying V numbers. The gradient of a loss $\mathcal{L}$ with respect to the matrix is

$$\frac{\partial \mathcal{L}}{\partial W} = \mathbf{e}_j \left(\frac{\partial \mathcal{L}}{\partial \mathbf{z}} \right)^\top, \quad \mathbf{z} = \mathbf{e}_j^\top W, \quad (17)$$

To see this, write $z_l = \sum_k (\mathbf{e}_j)_k W_{k,l}$, so that $\partial z_l / \partial W_{i,m} = (\mathbf{e}_j)_i \delta_{l,m}$, and use the chain rule:

$$\frac{\partial \mathcal{L}}{\partial W_{i,m}} = \sum_l \frac{\partial \mathcal{L}}{\partial z_l} (\mathbf{e}_j)_i \delta_{l,m} = (\mathbf{e}_j)_i \frac{\partial \mathcal{L}}{\partial z_m}, \quad (18)$$

the (i, m) entry of $\mathbf{e}_j (\partial \mathcal{L} / \partial \mathbf{z})^\top$. It is zero except in row j , so a plain gradient-descent step changes only the vector of the value that was seen (the optimiser used in the experiments, Adam, also moves other rows through its running averages). For a batch the gradients of the examples add. A test in the project checks the equality of a dense layer on one-hot input and an embedding layer numerically.

9.6 Comparing feature sets with paired seeds

Every feature set is trained with the same S seeds. For a pair of sets a and b and seed s , let $r_{a,s}$ and $r_{b,s}$ be the validation RMSE values (the tables of Section 6 use the validation set for the comparison). Because the same seed fixes the shuffling order and the random numbers that start the training, the errors of the two sets for the same seed are not independent, and the paired differences $d_s = r_{b,s} - r_{a,s}$ remove the part that they share. The mean and the sample standard deviation of the d_s are

$$\bar{d} = \frac{1}{S} \sum_{s=1}^S d_s, \quad s_d = \sqrt{\frac{1}{S-1} \sum_{s=1}^S (d_s - \bar{d})^2}, \quad (19)$$

and the 95% confidence interval for the mean difference is the interval of the Student t distribution with $S - 1$ degrees of freedom. Chapter 2 of the book applies the same function (`scipy.stats.t.interval`) to the squared errors of the test instances; here it is applied to the differences between seeds:

$$\bar{d} \pm t_{S-1, 0.975} \frac{s_d}{\sqrt{S}}. \quad (20)$$

A difference is called clear in the tables when this interval does not contain 0. The interval describes the variation caused by the seeds only. The data split is the same for all runs, so it does not include the variation that another split would add.

The interval of the test RMSE. Section 8 uses the function as Chapter 2 does. Let $q_i = (\hat{y}_i - y_i)^2$ be the squared error of test row i , for $i = 1, \dots, n$, with the mean $\bar{q}$ and the sample standard deviation s_q . The mean $\bar{q}$ is the mean squared error, so $\sqrt{\bar{q}}$ is the RMSE. Treat the q_i as independent draws with expectation μ , the mean squared error of the model on new rows. The 95% interval for μ is $[a, b]$ with

$$a, b = \bar{q} \mp t_{n-1, 0.975} \frac{s_q}{\sqrt{n}}. \quad (21)$$

The square root is increasing, so $a \leq \mu \leq b$ holds exactly when $\sqrt{a} \leq \sqrt{\mu} \leq \sqrt{b}$. The interval $[\sqrt{a}, \sqrt{b}]$ therefore contains $\sqrt{\mu}$ whenever $[a, b]$ contains μ , and it is a 95% interval for the RMSE:

$$\left[\sqrt{\bar{q} - t_{n-1, 0.975} s_q / \sqrt{n}}, \sqrt{\bar{q} + t_{n-1, 0.975} s_q / \sqrt{n}} \right]. \quad (22)$$

It is not symmetric about the RMSE, because the square root is not linear. The model is held fixed, so the interval describes only the variation that another sample of n test rows would cause.

10 Limits and what is not claimed

Machine and timings. All timings come from a cloud machine with two CPU cores and no GPU. Absolute numbers depend on the machine; ratios are what to compare. Each timing is the median of several passes after a warm-up pass, but two measurements of the same configuration still differ: within the run of this report the same CSV pipeline was timed twice and gave 15,104 and 16,602 examples per second (Section 4), 10% apart. Differences of that size or less between neighbouring rows of a table are not treated as effects. The tables of only one complete run are stored, so the variation between complete runs is not measured here.

Small data. The housing data has 20,640 rows and takes about 1.4 MB as a CSV file; the training rows alone take about 0.86 MB. An array in memory is faster than any of the streaming pipelines for such data, and the report shows it. The scale test repeats the training set up to 100 times; this tests how memory and speed of the pipeline grow, and no model is trained on the repeated rows. Even the largest repetition fits in the memory of the machine, so the report shows the different growth of memory but not a case in which pandas fails.

Features. The book's `tf.feature_column` API is replaced by Keras 3 preprocessing layers, so the models are not byte-for-byte the models of the book; in particular I did not check whether `HashedCrossing` assigns the same buckets as the old crossed column, and the collision counts of Section 6 are those of the layer used here. The comparison of feature sets uses one split of the data, so its intervals describe the variation caused by the seeds and not the variation another split would cause. The feature sets were not tuned: the sizes of the embeddings other than the book's own (2 dimensions for the categories) and the learning rates were set once.

MNIST. The version of TFDS used here downloads MNIST from a Google Cloud Storage address. When that address cannot be reached (some firewalls and CI runners cannot reach it, and this project was built in such an environment), the four official files are fetched from a public mirror, checked against the official MD5 checksums and passed to the MNIST builder of TFDS, so that everything after the download is the real `tfds.load`. The route that this run used is `mirror`. The two inputs that are compared in Part D are trained with the same classifier,

but the data from `tfds.load` are used in the order TFDS delivers them (the book says that the training set it returns is already shuffled), while the TFRecord pipeline shuffles with a buffer of 10,000 images. The comparison of their accuracies is a check that the TFRecord files contain the same data, not a measurement of which input is better.

Choices made after looking at results. The thresholds of the quality gates are choices of the project, set after looking at the results of a first run; they are meant to catch a broken model, not to define a good one. The feature set of the deployed model is chosen by a rule, the lowest mean validation RMSE of the neural network in Part C. The first version of the configuration named the set `all features` instead; after the comparison showed that this set was not better than the baseline, the configuration was changed to the rule and the deployment was repeated. The test set played no part in this choice. It is used, however, by the quality gates and by the comparison with the champion, and the thresholds of the gates were set after a first run had been evaluated on it. Chapter 1 of the book warns that a model adapted to a test set by repeated measurements is unlikely to do as well on new data. The test numbers of this report are therefore a final report of this run, not a fresh estimate that no decision has seen.

Sources. The data loading and preprocessing follow Chapter 13 of the book and the housing data, the split and the confidence interval follow its Chapter 2. The MLOps system around them (Prefect, MLflow, FastAPI, PostgreSQL, Docker Compose, the quality gates, the monitoring and the continuous integration) is not in the book; it follows the structure of the example repository cited in the bibliography.

10.1 What I learned and what I would do next

Most of what I learned is about measuring. A step of a pipeline has to be timed with the automatic optimisations of `tf.data` switched off as well as on, because the optimisations hid most of the effect of the book's steps. One statistic can mislead: reading the files in random order gave an order correlation near 0 while every batch still held almost the same targets. A comparison of two formats needs a control: the TFRecord pipeline differed from the CSV pipeline in the format and in the batching at once. And timings from one machine are compared as ratios, with differences of about 10% treated as noise.

The next steps follow from what this report did not separate or did not test:

- a CSV reader that parses a whole batch of lines in one call, to separate the effect of the TFRecord format from that of batch-wise parsing (Section 5);
- the comparison of `AUTOTUNE` with the book's fixed values on a machine with more cores and with longer passes, since the cause of the slower `AUTOTUNE` runs was not investigated (Sections 4 and 7);
- the comparison of feature sets on other splits of the data, so that the intervals include the variation between splits and not only that between seeds (Section 6);
- a test of the explanation, so far untested, of why the network gains little from the grid (that its hidden layers build functions of position from raw latitude and longitude), and an experiment that isolates the effect of the hash collisions (Section 6);
- data that do not fit in memory, which the scale test, in which every repetition fits comfortably, did not reach (Section 5).

11 Reproducing the results

The commands are run from the root of the repository. The quick configuration trains on the same data with fewer repetitions and takes a few minutes. It writes its results to `reports/quick/`, so the committed results of the full run stay as they are. The full configuration, whose results are in this report, took about 45 minutes on the two-core machine that produced it.

```
make install    # Python 3.12 or newer
make test      # offline tests
make quick     # quick configuration; writes reports/quick/
make train     # full experiment; writes the tables, figures and RESULTS.md
make report    # tables of the last run to LaTeX, then this PDF
make up        # the whole stack in Docker (PostgreSQL, MLflow, Prefect, API)
```

The run that produced this report used the configuration `configs/full_flow_config.yaml` with seed 42 on the data with SHA-256 prefix 8a3727f4cf54.

References

- [1] A. Géron, *Hands-On Machine Learning with Scikit-Learn, Keras and TensorFlow*, 2nd edition, O'Reilly, 2019. Chapter 1 (the use of the test set), Chapter 2 (the housing data, the split, the confidence interval, the capped values) and Chapter 13 (the Data API, the TFRecord format, the Features API and TensorFlow Datasets).
- [2] GitHub user jomariya23156, *full-stack-on-prem-cv-mlops*, public repository, <https://github.com/jomariya23156/full-stack-on-prem-cv-mlops>. The structure of the surrounding MLOps system (one configuration file, train, evaluate and deploy flows, a service that serves the registered model) follows this project.